2026-05-05-p1-f05-hook-based-extensibility

P1-F05 — Hook-Based Extensibility Implementation Plan

> **For agentic workers:** REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (`- [ ]`) syntax for tracking.

Goal: Implement claude-code-style hook-based extensibility: users author `~/.helixcode/hooks.yaml` (and optionally `<project>/.helixcode/hooks.yaml`) listing event-type → shell-script mappings; HelixCode loads them at startup and fires them at 9 documented lifecycle points across `tools/registry.Execute`, `llm/compression/AutoCompactor`, and `agent/agent.go`.

Architecture: Extend the existing `internal/hooks/` package (already has Manager, Hook, Event, Executor, sync/async/wait Trigger* methods). Add 6 new HookType constants + 3 new files (`yaml_loader.go`, `shell_runner.go`, `blockers.go`). Wire dispatch into 4 packages. Reuse `session.Manager.GetHooksManager()` to share one Manager instance across the program.

Tech Stack: Go 1.26, testify v1.11, github.com/spf13/cobra v1.8, gopkg.in/yaml.v3 (already in go.mod), existing `internal/hooks/`. **No new dependencies.** Standard-library `os/exec`, `encoding/json`, `path/filepath`, `regexp`, `time`, `sync`.

Spec: `docs/superpowers/specs/2026-05-05-p1-f05-hook-based-extensibility-design.md` (commit `118df80`)

Working directory for all `go` commands: `helix_code/` (the inner Go module). Git commands run from the meta-repo root `/run/media/milosvasic/DATA4TB/Projects/helix_code/` per the F01–F04 convention.

Anti-bluff smoke (run on every commit, FULL pattern):

```
cd HelixCode && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/hooks/ && echo "BLUFF FOUND" ||
        echo "clean"
```

Task 1: Bootstrap evidence + advance PROGRESS

Files: - Modify: `docs/improvements/06_phase_1_evidence.md` - Modify: `docs/improvements/PROGRESS.md`

- [ ] **Step 1: Append F05 section header to evidence file**

Append (do NOT overwrite) to `docs/improvements/06_phase_1_evidence.md`:

```
---

## P1-F05 — Hook-Based Extensibility

**Spec:** `docs/superpowers/specs/2026-05-05-p1-f05-hook-based-
        extensibility-design.md` (commit `118df80`)
**Plan:** `docs/superpowers/plans/2026-05-05-p1-f05-hook-based-
        extensibility.md`
**Started:** 2026-05-05
**Status:** active

### Task evidence trail

(filled in commit-by-commit as tasks land)
```

- [ ] **Step 2: Update PROGRESS.md current focus block**

Replace the existing "## Current focus" block with:

```
## Current focus
- **Active phase:** P1 — claude-code feature porting
- **Active feature:** F05 — Hook-Based Extensibility
- **Active task:** P1-F05-T01 — bootstrap evidence + advance
        PROGRESS
- **Last completed:** P1-F04-T13 — Feature 4 (Git Worktree Agent
        Isolation) close-out + push
- **Owner:** agent (Claude Opus 4.7)
- **Started:** 2026-05-04
- **Last touched:** 2026-05-05
- **Blocked-on:** none
```

- [ ] **Step 3: Add F05 task list block to PROGRESS.md**

After the existing F04 task list block (all 13 items checked), insert:

```
## Active feature task list (P1-F05: Hook-Based Extensibility)
- [ ] P1-F05-T01 — bootstrap evidence + advance PROGRESS
- [ ] P1-F05-T02 — add 6 new HookType constants (TDD)
- [ ] P1-F05-T03 — yaml_loader.go: FileLoader + apiVersion
        validation (TDD)
- [ ] P1-F05-T04 — shell_runner.go: NewShellRunner HookFunc (TDD)
- [ ] P1-F05-T05 — blockers.go: Blockers helper (TDD)
- [ ] P1-F05-T06 — wire registry.Execute with 6 events (TDD)
- [ ] P1-F05-T07 — wire OnCompaction in AutoCompactor (TDD)
- [ ] P1-F05-T08 — wire OnError + RequestPlanApproval stub in
        agent.go (TDD)
```

```
- [ ] P1-F05-T09 — helixcode hooks
        {list,test,enable,disable,validate} subcommands
- [ ] P1-F05-T10 — /hooks slash command + builtin registration
- [ ] P1-F05-T11 — cmd/cli/main.go startup wiring + integration
        tests (no mocks)
- [ ] P1-F05-T12 — Challenge with three runtime-evidence
        scenarios
- [ ] P1-F05-T13 — Feature 5 close-out + push
```

- [ ] **Step 4: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add docs/
        improvements/06_phase_1_evidence.md docs/improvements/
        PROGRESS.md
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
docs(P1-F05-T01): bootstrap Phase 1 / Feature 5 evidence +
        advance PROGRESS

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 2: Add 6 new HookType constants (TDD)

Files: - Modify: `helix_code/internal/hooks/hook.go` (extend the existing const block at lines 12-25) - Create: `helix_code/internal/hooks/hook_types_p1f05_test.go`

- [ ] **Step 1: Write failing test**

Create `helix_code/internal/hooks/hook_types_p1f05_test.go`:

```
package hooks

import (
    "testing"

    "github.com/stretchr/testify/assert"
)

// TestF05HookTypes_AreDistinct ensures the 6 new HookType
        constants for F05
// don't collide with each other or with existing constants.
func TestF05HookTypes_AreDistinct(t *testing.T) {
```

```
    newTypes := []HookType{
        HookTypeBeforeToolCall,
        HookTypeAfterToolCall,
        HookTypeBeforeBash,
        HookTypeAfterBash,
        HookTypeOnCompaction,
        HookTypeOnPlanApproval,
    }
    seen := map[HookType]bool{}
    for _, ht := range newTypes {
        assert.NotEmpty(t, string(ht), "HookType must have a non-
        empty string value")
        assert.False(t, seen[ht], "duplicate HookType value:
        %q", ht)
        seen[ht] = true
    }
}

// TestF05HookTypes_StringValues asserts each new HookType
       serialises to the
// canonical string identifier used in YAML / wire format.
func TestF05HookTypes_StringValues(t *testing.T) {
    cases := map[HookType]string{
        HookTypeBeforeToolCall: "before_tool_call",
        HookTypeAfterToolCall:  "after_tool_call",
        HookTypeBeforeBash:     "before_bash",
        HookTypeAfterBash:      "after_bash",
        HookTypeOnCompaction:   "on_compaction",
        HookTypeOnPlanApproval: "on_plan_approval",
    }
    for ht, expected := range cases {
        assert.Equal(t, expected, string(ht), "HookType %q has
        wrong string value", expected)
    }
}

// TestF05HookTypes_DoNotCollideWithExisting ensures the new
       identifiers
// don't shadow existing claude-code-mismatched event names.
func TestF05HookTypes_DoNotCollideWithExisting(t *testing.T) {
    existing := []HookType{
        HookTypeBeforeTask, HookTypeAfterTask,
        HookTypeBeforeLLM, HookTypeAfterLLM,
        HookTypeBeforeEdit, HookTypeAfterEdit,
        HookTypeBeforeBuild, HookTypeAfterBuild,
        HookTypeBeforeTest, HookTypeAfterTest,
        HookTypeOnError, HookTypeOnSuccess,
        HookTypeCustom,
```

```
    }
    new := []HookType{
        HookTypeBeforeToolCall, HookTypeAfterToolCall,
        HookTypeBeforeBash, HookTypeAfterBash,
        HookTypeOnCompaction, HookTypeOnPlanApproval,
    }
    all := append([]HookType{}, existing...)
    all = append(all, new...)
    seen := map[HookType]bool{}
    for _, ht := range all {
        assert.False(t, seen[ht],
        "HookType collision: %q already exists", ht)
        seen[ht] = true
    }
}
```

☐ **Step 2: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run 'TestF05HookTypes' ./internal/
        hooks/
```

Expected: FAIL — the new HookType constants are undefined.

☐ **Step 3: Add the new constants to hook.go**

In `helix_code/internal/hooks/hook.go`, find the existing `const ( … HookType ... )` block (around lines 12-25). Add the 6 new constants at the end of the block, BEFORE the closing `)`:

```
const (
    HookTypeBeforeTask  HookType = "before_task"
    HookTypeAfterTask   HookType = "after_task"
    HookTypeBeforeLLM   HookType = "before_llm"
    HookTypeAfterLLM    HookType = "after_llm"
    HookTypeBeforeEdit  HookType = "before_edit"
    HookTypeAfterEdit   HookType = "after_edit"
    HookTypeBeforeBuild HookType = "before_build"
    HookTypeAfterBuild  HookType = "after_build"
    HookTypeBeforeTest  HookType = "before_test"
    HookTypeAfterTest   HookType = "after_test"
    HookTypeOnError     HookType = "on_error"
    HookTypeOnSuccess   HookType = "on_success"
    HookTypeCustom      HookType = "custom"

    // P1-F05 additions: claude-code-style lifecycle events.
    HookTypeBeforeToolCall HookType = "before_tool_call"
    HookTypeAfterToolCall  HookType = "after_tool_call"
```

```
    HookTypeBeforeBash     HookType = "before_bash"
    HookTypeAfterBash      HookType = "after_bash"
    HookTypeOnCompaction   HookType = "on_compaction"
    HookTypeOnPlanApproval HookType = "on_plan_approval"
)
```

- [ ] **Step 4: Run tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run 'TestF05HookTypes' ./
        internal/hooks/
```

Expected: PASS — 3 tests.

- [ ] **Step 5: Run full hooks package (regression check)**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race ./internal/hooks/...
```

Expected: PASS — no regression in existing hook tests.

- [ ] **Step 6: Anti-bluff smoke (FULL pattern)**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/hooks/hook.go internal/hooks/
        hook_types_p1f05_test.go && echo "BLUFF FOUND" || echo
        "clean"
```

Expected: `clean`. (Existing hook.go may have pre-existing hits — only flag NEW lines.)

- [ ] **Step 7: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/hooks/hook.go helix_code/internal/
        hooks/hook_types_p1f05_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F05-T02): add 6 new HookType constants for claude-code
        lifecycle events

HookTypeBeforeToolCall, AfterToolCall, BeforeBash, AfterBash,
OnCompaction, OnPlanApproval. Existing BeforeEdit, AfterEdit,
        OnError
are reused per the spec. 3 unit tests verify distinct string
        values,
```

```
canonical identifiers (before_tool_call etc.), and no collision
        with
the 13 pre-existing types.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 3: yaml_loader.go — FileLoader with apiVersion validation (TDD)

Files: - Create: `helix_code/internal/hooks/yaml_loader.go` - Create: `helix_code/internal/hooks/yaml_loader_test.go`

- [ ] **Step 1: Write failing test**

Create `helix_code/internal/hooks/yaml_loader_test.go`:

```
package hooks

import (
    "context"
    "os"
    "path/filepath"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

func TestFileLoader_BothFilesMissing(t *testing.T) {
    tmp := t.TempDir()
    loader := &FileLoader{
        UserPath:    filepath.Join(tmp, "user.yaml"),
        ProjectPath: filepath.Join(tmp, "project.yaml"),
    }
    hooks, sources, err := loader.Load(context.Background())
    require.NoError(t, err)
    assert.Empty(t, hooks)
    assert.Empty(t, sources)
}

func TestFileLoader_UserFileOnly(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
```

```
    scriptPath := filepath.Join(tmp, "audit.sh")
    require.NoError(t, os.WriteFile(scriptPath, []byte("#!/bin/
        sh\nexit 0\n"), 0o755))
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.hooks/v1
hooks:
  - id: audit
    event: before_tool_call
    script: `+scriptPath+`
    priority: 100
    enabled: true
`), 0o600))
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        filepath.Join(tmp, "missing.yaml")}
    hooks, sources, err := loader.Load(context.Background())
    require.NoError(t, err)
    require.Len(t, hooks, 1)
    assert.Equal(t, "audit", hooks[0].ID)
    assert.Equal(t, HookTypeBeforeToolCall, hooks[0].Type)
    assert.Equal(t, HookPriority(100), hooks[0].Priority)
    assert.True(t, hooks[0].Enabled)
    assert.Equal(t, []string{userPath}, sources)
}

func TestFileLoader_ProjectOverridesUserSameID(t *testing.T) {
    tmp := t.TempDir()
    scriptA := filepath.Join(tmp, "a.sh")
    scriptB := filepath.Join(tmp, "b.sh")
    require.NoError(t, os.WriteFile(scriptA, []byte("#!/bin/
        sh\nexit 0\n"), 0o755))
    require.NoError(t, os.WriteFile(scriptB, []byte("#!/bin/
        sh\nexit 0\n"), 0o755))

    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.hooks/v1
hooks:
  - id: dup
    event: before_bash
    script: `+scriptA+`
    priority: 1
`), 0o600))

    projPath := filepath.Join(tmp, "project.yaml")
    require.NoError(t, os.WriteFile(projPath,
        []byte(`apiVersion: helixcode.hooks/v1
hooks:
  - id: dup
```

```
    event: before_bash
    script: `+scriptB+`
    priority: 999
`), 0o600))

    loader := &FileLoader{UserPath: userPath, ProjectPath:
        projPath}
    hooks, _, err := loader.Load(context.Background())
    require.NoError(t, err)
    require.Len(t, hooks, 1, "duplicate id collapses to one
        entry")
    assert.Equal(t, HookPriority(999), hooks[0].Priority,
        "project overrides user")
}

func TestFileLoader_DisabledHooksAreFiltered(t *testing.T) {
    tmp := t.TempDir()
    scriptPath := filepath.Join(tmp, "x.sh")
    require.NoError(t, os.WriteFile(scriptPath, []byte("#!/bin/
        sh\nexit 0\n"), 0o755))
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.hooks/v1
hooks:
  - id: on
    event: on_error
    script: `+scriptPath+`
    enabled: true
  - id: off
    event: on_error
    script: `+scriptPath+`
    enabled: false
`), 0o600))
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        filepath.Join(tmp, "missing.yaml")}
    hooks, _, err := loader.Load(context.Background())
    require.NoError(t, err)
    require.Len(t, hooks, 1, "disabled hooks must not be
        returned")
    assert.Equal(t, "on", hooks[0].ID)
}

func TestFileLoader_MalformedYAMLIsError(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte("not: valid: yaml: ["), 0o600))
```

```
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        filepath.Join(tmp, "missing.yaml")}
    _, _, err := loader.Load(context.Background())
    assert.Error(t, err)
}

func TestFileLoader_MissingAPIVersionIsError(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath, []byte(`hooks:
  - id: x
    event: on_error
    script: /bin/true
`), 0o600))
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        filepath.Join(tmp, "missing.yaml")}
    _, _, err := loader.Load(context.Background())
    require.Error(t, err)
    assert.Contains(t, err.Error(), "apiVersion")
}

func TestFileLoader_UnknownAPIVersionIsError(t *testing.T) {
    tmp := t.TempDir()
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.hooks/v999
hooks: []
`), 0o600))
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        filepath.Join(tmp, "missing.yaml")}
    _, _, err := loader.Load(context.Background())
    require.Error(t, err)
    assert.Contains(t, err.Error(), "unsupported apiVersion")
}

func TestFileLoader_UnknownEventTypeRejectedAtLoad(t *testing.T)
        {
    tmp := t.TempDir()
    scriptPath := filepath.Join(tmp, "x.sh")
    require.NoError(t, os.WriteFile(scriptPath, []byte("#!/bin/
        sh\nexit 0\n"), 0o755))
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.hooks/v1
hooks:
  - id: bad
    event: nonsense_event
    script: `+scriptPath+`
```

```
  - id: good
    event: before_tool_call
    script: `+scriptPath+`
`), 0o600))
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        filepath.Join(tmp, "missing.yaml")}
    hooks, _, err := loader.Load(context.Background())
    require.NoError(t, err)
    require.Len(t, hooks, 1, "unknown event types are skipped;
        valid hooks still load")
    assert.Equal(t, "good", hooks[0].ID)
}

func TestFileLoader_TimeoutParsesGoDuration(t *testing.T) {
    tmp := t.TempDir()
    scriptPath := filepath.Join(tmp, "x.sh")
    require.NoError(t, os.WriteFile(scriptPath, []byte("#!/bin/
        sh\nexit 0\n"), 0o755))
    userPath := filepath.Join(tmp, "user.yaml")
    require.NoError(t, os.WriteFile(userPath,
        []byte(`apiVersion: helixcode.hooks/v1
hooks:
  - id: slow
    event: before_tool_call
    script: `+scriptPath+`
    timeout: 5s
`), 0o600))
    loader := &FileLoader{UserPath: userPath, ProjectPath:
        filepath.Join(tmp, "missing.yaml")}
    hooks, _, err := loader.Load(context.Background())
    require.NoError(t, err)
    require.Len(t, hooks, 1)
    assert.Equal(t, 5*time.Second, hooks[0].Timeout)
}
```

- [ ] **Step 2: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run 'TestFileLoader' ./internal/
        hooks/
```

Expected: FAIL — `FileLoader` undefined.

- [ ] **Step 3: Implement yaml_loader.go**

Create `helix_code/internal/hooks/yaml_loader.go`:

```go
package hooks

import (
    "context"
    "errors"
    "fmt"
    "log"
    "os"
    "strings"
    "time"

    "gopkg.in/yaml.v3"
)

// expectedAPIVersion is the only YAML schema version this loader
        accepts.
const expectedAPIVersion = "helixcode.hooks/v1"

// fileSchema is the on-disk YAML structure for hooks files.
type fileSchema struct {
    APIVersion string       `yaml:"apiVersion"`
    Hooks      []hookSchema `yaml:"hooks"`
}

type hookSchema struct {
    ID          string `yaml:"id"`
    Event       string `yaml:"event"`
    Script      string `yaml:"script"`
    Priority    int    `yaml:"priority"`
    Async       bool   `yaml:"async"`
    Timeout     string `yaml:"timeout"`
    Enabled     *bool  `yaml:"enabled"` // pointer so absent ≠
        false
    Description string `yaml:"description"`
}

// FileLoader reads hooks from layered YAML files.
//
// Project file entries override user file entries with the same
        id.
// Disabled hooks are filtered out before return.
type FileLoader struct {
    UserPath    string
    ProjectPath string
}

// Load reads both files and returns the aggregated, enabled
        hooks plus
```

```
// the source paths in load order. Missing files are not errors.
func (l *FileLoader) Load(ctx context.Context) ([]*Hook,
        []string, error) {
    user, err := readFileIfExists(l.UserPath)
    if err != nil {
        return nil, nil, fmt.Errorf("reading user file %s: %w",
        l.UserPath, err)
    }
    project, err := readFileIfExists(l.ProjectPath)
    if err != nil {
        return nil, nil, fmt.Errorf("reading project file %s:
        %w", l.ProjectPath, err)
    }

    var sources []string
    if user != nil {
        if err := validateAPIVersion(user.APIVersion); err !=
        nil {
            return nil, nil, fmt.Errorf("%s: %w", l.UserPath,
        err)
        }
        sources = append(sources, l.UserPath)
    }
    if project != nil {
        if err := validateAPIVersion(project.APIVersion); err !=
        nil {
            return nil, nil, fmt.Errorf("%s: %w", l.ProjectPath,
        err)
        }
        sources = append(sources, l.ProjectPath)
    }

    merged := mergeHooks(project, user)
    return filterEnabledAndConvert(merged), sources, nil
}

func readFileIfExists(path string) (*fileSchema, error) {
    if path == "" {
        return nil, nil
    }
    body, err := os.ReadFile(path)
    if err != nil {
        if errors.Is(err, os.ErrNotExist) {
            return nil, nil
        }
        return nil, err
    }
    var f fileSchema
```

```
    if err := yaml.Unmarshal(body, &f); err != nil {
        return nil, fmt.Errorf("yaml: %w", err)
    }
    return &f, nil
}

func validateAPIVersion(v string) error {
    if v == "" {
        return fmt.Errorf("missing apiVersion (expected %q)",
        expectedAPIVersion)
    }
    if v != expectedAPIVersion {
        return fmt.Errorf("unsupported apiVersion %q (expected
        %q)", v, expectedAPIVersion)
    }
    return nil
}

// mergeHooks merges project + user files. Identical IDs: project
        wins.
func mergeHooks(project, user *fileSchema) []hookSchema {
    var merged []hookSchema
    projectIDs := map[string]bool{}
    if project != nil {
        for _, h := range project.Hooks {
            merged = append(merged, h)
            projectIDs[h.ID] = true
        }
    }
    if user != nil {
        for _, h := range user.Hooks {
            if projectIDs[h.ID] {
                continue
            }
            merged = append(merged, h)
        }
    }
    return merged
}

// filterEnabledAndConvert turns hookSchema into *Hook entries,
        dropping
// disabled hooks and entries whose event type is unknown
        (logged, skipped).
func filterEnabledAndConvert(schemas []hookSchema) []*Hook {
    out := make([]*Hook, 0, len(schemas))
    for _, s := range schemas {
        enabled := true
```

```
        if s.Enabled != nil {
            enabled = *s.Enabled
        }
        if !enabled {
            continue
        }
        evt, ok := parseEventType(s.Event)
        if !ok {
            log.Printf("WARN hooks loader: unknown event type %q
        in hook %q — skipping", s.Event, s.ID)
            continue
        }
        var timeout time.Duration
        if strings.TrimSpace(s.Timeout) != "" {
            d, err := time.ParseDuration(s.Timeout)
            if err != nil {

        log.Printf("WARN hooks loader: invalid timeout %q in hook
        %q — using 0", s.Timeout, s.ID)
            } else {
                timeout = d
            }
        }
        hook := &Hook{
            ID:          s.ID,
            Name:        s.ID, // use id as display name; users
        can override later
            Type:        evt,
            Description: s.Description,
            Priority:    HookPriority(s.Priority),
            Async:       s.Async,
            Timeout:     timeout,
            Enabled:     true,
            CreatedAt:   time.Now(),
            Metadata:    map[string]string{"script": s.Script},
        }
        out = append(out, hook)
    }
    return out
}

// parseEventType resolves a YAML event-type string to a
        HookType, returning
// false if unknown. Covers all 19 known constants (13 pre-
        existing + 6 F05).
func parseEventType(s string) (HookType, bool) {
    switch HookType(s) {
    case HookTypeBeforeTask, HookTypeAfterTask,
```

```
        HookTypeBeforeLLM, HookTypeAfterLLM,
        HookTypeBeforeEdit, HookTypeAfterEdit,
        HookTypeBeforeBuild, HookTypeAfterBuild,
        HookTypeBeforeTest, HookTypeAfterTest,
        HookTypeOnError, HookTypeOnSuccess,
        HookTypeCustom,
        HookTypeBeforeToolCall, HookTypeAfterToolCall,
        HookTypeBeforeBash, HookTypeAfterBash,
        HookTypeOnCompaction, HookTypeOnPlanApproval:
        return HookType(s), true
    }
    return "", false
}
```

- [ ] **Step 4: Run tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run 'TestFileLoader' ./
        internal/hooks/
```

Expected: PASS — 9 tests.

- [ ] **Step 5: Run full hooks package**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race ./internal/hooks/...
```

Expected: PASS — no regression.

- [ ] **Step 6: Anti-bluff smoke (FULL pattern)**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/hooks/yaml_loader.go internal/
        hooks/yaml_loader_test.go && echo "BLUFF FOUND" || echo
        "clean"
```

Expected: `clean`.

- [ ] **Step 7: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/hooks/yaml_loader.go helix_code/
        internal/hooks/yaml_loader_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
```

```
feat(P1-F05-T03): yaml_loader.go — FileLoader with apiVersion
        validation (TDD)

FileLoader reads ~/.helixcode/hooks.yaml + <project>/.helixcode/
        hooks.yaml,
validates apiVersion: helixcode.hooks/v1, merges entries (project
overrides user on identical id), drops disabled hooks, and skips
        entries
with unknown event types (logged at WARN).
        Hook.Metadata["script"]
carries the script path for the shell-runner to consume in T04.
9 unit tests including the both-files-missing case, project
        override,
malformed YAML, missing/unknown apiVersion, and unknown event
        type
rejection at load.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 4: shell_runner.go — NewShellRunner HookFunc (TDD)

Files: - Create: `helix_code/internal/hooks/shell_runner.go` - Create: `helix_code/internal/hooks/shell_runner_test.go`

- [ ] **Step 1: Write failing test**

Create `helix_code/internal/hooks/shell_runner_test.go`:

```go
package hooks

import (
    "context"
    "errors"
    "os"
    "path/filepath"
    "testing"
    "time"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"
)

// writeScript creates a temp shell script and returns its path.
```

```
func writeScript(t *testing.T, body string) string {
    t.Helper()
    tmp := t.TempDir()
    path := filepath.Join(tmp, "hook.sh")
    require.NoError(t, os.WriteFile(path, []byte("#!/bin/
        sh\n"+body+"\n"), 0o755))
    return path
}

func TestShellRunner_ExitZeroIsSuccess(t *testing.T) {
    script := writeScript(t, "exit 0")
    runner := NewShellRunner(script, 0)
    event := NewEvent(HookTypeBeforeToolCall)
    event.SetData("toolName", "Bash")
    err := runner(context.Background(), event)
    assert.NoError(t, err)
}

func TestShellRunner_NonZeroExitIsBlock(t *testing.T) {
    script := writeScript(t, "echo 'blocked!' >&2; exit 1")
    runner := NewShellRunner(script, 0)
    event := NewEvent(HookTypeBeforeBash)
    err := runner(context.Background(), event)
    require.Error(t, err)
    assert.Contains(t, err.Error(), "blocked!", "stderr must
        surface in error")
}

func TestShellRunner_TimeoutAborts(t *testing.T) {
    script := writeScript(t, "sleep 5")
    runner := NewShellRunner(script, 100*time.Millisecond)
    event := NewEvent(HookTypeOnError)
    start := time.Now()
    err := runner(context.Background(), event)
    elapsed := time.Since(start)
    require.Error(t, err)
    assert.Less(t, elapsed, 2*time.Second, "timeout must fire
        well before 5s sleep")
}

func TestShellRunner_MissingScriptIsBlock(t *testing.T) {
    runner := NewShellRunner("/nonexistent/script.sh", 0)
    event := NewEvent(HookTypeOnCompaction)
    err := runner(context.Background(), event)
    require.Error(t, err)
}

func TestShellRunner_StdinReceivesEventJSON(t *testing.T) {
```

```
    tmp := t.TempDir()
    stdinCapture := filepath.Join(tmp, "captured.json")
    script := writeScript(t, "cat > "+stdinCapture)
    runner := NewShellRunner(script, 0)
    event := NewEvent(HookTypeBeforeToolCall)
    event.Source = "tool_registry"
    event.SetData("toolName", "Bash")
    event.SetData("params", map[string]interface{}{"command":
        "ls"})
    require.NoError(t, runner(context.Background(), event))

    body, err := os.ReadFile(stdinCapture)
    require.NoError(t, err)
    assert.Contains(t, string(body), `"type":"before_tool_call"`)
    assert.Contains(t, string(body), `"toolName":"Bash"`)
    assert.Contains(t, string(body), `"command":"ls"`)
}

func TestShellRunner_StdoutModifyPayloadMergedIntoEvent(t
        *testing.T) {
    // Script prints a JSON object on stdout that the runner
        merges into event.Data.
    script := writeScript(t, `echo '{"data":
        {"injected":"value"}}'`)
    runner := NewShellRunner(script, 0)
    event := NewEvent(HookTypeBeforeToolCall)
    event.SetData("original", "x")
    require.NoError(t, runner(context.Background(), event))
    assert.Equal(t, "x", event.Data["original"], "original keys
        preserved")
    assert.Equal(t, "value", event.Data["injected"],
        "stdout JSON merged into event.Data")
}

func TestShellRunner_MalformedStdoutIsLoggedNotBlock(t
        *testing.T) {
    script := writeScript(t, `echo 'this is not json'; exit 0`)
    runner := NewShellRunner(script, 0)
    event := NewEvent(HookTypeBeforeToolCall)
    err := runner(context.Background(), event)
    assert.NoError(t, err,
        "malformed stdout JSON must NOT block; only logged")
}

func TestShellRunner_RespectsCallerContextCancel(t *testing.T) {
    script := writeScript(t, "sleep 5")
    runner := NewShellRunner(script, 10*time.Second)
    ctx, cancel := context.WithCancel(context.Background())
```

```
    go func() {
        time.Sleep(100 * time.Millisecond)
        cancel()
    }()
    event := NewEvent(HookTypeOnError)
    start := time.Now()
    err := runner(ctx, event)
    elapsed := time.Since(start)
    require.Error(t, err)
    assert.True(t, errors.Is(err, context.Canceled) || elapsed <
        2*time.Second,
        "caller cancel must abort run; got err=%v elapsed=%s",
        err, elapsed)
}
```

- [ ] **Step 2: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run 'TestShellRunner' ./internal/
        hooks/
```

Expected: FAIL — `NewShellRunner` undefined.

- [ ] **Step 3: Implement shell_runner.go**

Create `helix_code/internal/hooks/shell_runner.go`:

```
package hooks

import (
    "bytes"
    "context"
    "encoding/json"
    "fmt"
    "log"
    "os/exec"
    "time"
)

// shellRunnerPayload is the JSON document written to a hook
        script's stdin.
type shellRunnerPayload struct {
    Type      string                 `json:"type"`
    Timestamp string                 `json:"timestamp"`
    SessionID string
        `json:"session_id,omitempty"`
    Source    string                 `json:"source,omitempty"`
    Data      map[string]interface{} `json:"data"`
```

```
}

// shellRunnerModify is the JSON document a hook may print to
        stdout to mutate
// event.Data for downstream handlers. Read in F05; back-
        propagation to
// originating-operation params is N1 (out of scope).
type shellRunnerModify struct {
    Data map[string]interface{} `json:"data"`
}

// NewShellRunner returns a HookFunc that exec's scriptPath with
        the event
// payload on stdin. Behaviour:
//   - Non-zero exit → returns error wrapping captured stderr
        (treated as block).
//   - timeout > 0 → applied via context.WithTimeout; deadline =
        block.
//   - Missing script → returns error (fail-closed).
//   - Stdout JSON matching {"data":{...}} → merged into
        event.Data for
//     downstream handlers; malformed stdout JSON is logged and
        ignored.
//   - Caller's context cancellation aborts the script.
func NewShellRunner(scriptPath string, timeout time.Duration)
        HookFunc {
    return func(ctx context.Context, event *Event) error {
        runCtx := ctx
        var cancel context.CancelFunc
        if timeout > 0 {
            runCtx, cancel = context.WithTimeout(ctx, timeout)
            defer cancel()
        }

        payload := shellRunnerPayload{
            Type:      string(event.Type),
            Timestamp: event.Timestamp.Format(time.RFC3339),
            Source:    event.Source,
            Data:      event.Data,
        }
        if sid, ok := event.Metadata["session_id"]; ok {
            payload.SessionID = sid
        }
        stdinJSON, err := json.Marshal(payload)
        if err != nil {
            return fmt.Errorf("marshalling event payload: %w",
        err)
        }
```

```
        cmd := exec.CommandContext(runCtx, scriptPath)
        cmd.Stdin = bytes.NewReader(stdinJSON)
        var stdout, stderr bytes.Buffer
        cmd.Stdout = &stdout
        cmd.Stderr = &stderr

        runErr := cmd.Run()
        if runErr != nil {
            // Distinguish missing-script / context-cancelled /
        non-zero-exit
            // for the caller's diagnostic surface.
            return fmt.Errorf("hook script %s: %w (stderr: %s)",
        scriptPath, runErr, stderr.String())
        }

        // Read modify-payload from stdout. Malformed = log +
        ignore.
        if stdout.Len() > 0 {
            var mod shellRunnerModify
            if jerr := json.Unmarshal(stdout.Bytes(), &mod);
        jerr != nil {
                log.Printf("WARN hooks shell_runner: stdout from
        %s is not valid JSON; ignoring (%v)", scriptPath, jerr)
            } else if mod.Data != nil {
                if event.Data == nil {
                    event.Data = map[string]interface{}{}
                }
                for k, v := range mod.Data {
                    event.Data[k] = v
                }
            }
        }
        return nil
    }
}
```

- [ ] **Step 4: Run tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run 'TestShellRunner' ./
        internal/hooks/
```

Expected: PASS — 8 tests.

- [ ] **Step 5: Run full hooks package**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race ./internal/hooks/...
```

Expected: PASS — no regression.

- [ ] **Step 6: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/hooks/shell_runner.go internal/
        hooks/shell_runner_test.go && echo "BLUFF FOUND" || echo
        "clean"
```

Expected: `clean`.

- [ ] **Step 7: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/hooks/shell_runner.go helix_code/
        internal/hooks/shell_runner_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F05-T04): shell_runner.go — NewShellRunner HookFunc (TDD)

NewShellRunner returns a HookFunc that exec's the script with the
        event
payload as JSON on stdin. Non-zero exit → blocking error wrapping
        stderr.
timeout > 0 → context.WithTimeout. Missing script → fail-closed.
Stdout JSON of shape {"data":{...}} → merged into event.Data for
downstream handlers (F05 N1: back-propagation to operation params
        is
out of scope). Caller context cancel aborts the run. 8 unit tests
against real bash scripts in t.TempDir().

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 5: blockers.go — Blockers helper (TDD)

Files: - Create: `helix_code/internal/hooks/blockers.go` - Create: `helix_code/internal/hooks/blockers_test.go`

- [ ] **Step 1: Write failing test**

Create `helix_code/internal/hooks/blockers_test.go`:

```
package hooks

import (
    "errors"
    "testing"

    "github.com/stretchr/testify/assert"
)

func TestBlockers_NilSlice(t *testing.T) {
    assert.Nil(t, Blockers(nil))
}

func TestBlockers_AllSucceeded(t *testing.T) {
    results := []*ExecutionResult{
        {HookID: "a", Status: StatusSucceeded, Error: nil},
        {HookID: "b", Status: StatusSucceeded, Error: nil},
    }
    assert.Empty(t, Blockers(results))
}

func TestBlockers_OneFailed(t *testing.T) {
    results := []*ExecutionResult{
        {HookID: "a", Status: StatusSucceeded, Error: nil},
        {HookID: "b", Status: StatusFailed, Error:
        errors.New("nope")},
    }
    got := Blockers(results)
    assert.Len(t, got, 1)
    assert.Contains(t, got[0].Error(), "nope")
}

func TestBlockers_MultipleFailed_PreservesOrder(t *testing.T) {
    results := []*ExecutionResult{
        {HookID: "a", Status: StatusFailed, Error:
        errors.New("first")},
        {HookID: "b", Status: StatusSucceeded, Error: nil},
        {HookID: "c", Status: StatusFailed, Error:
        errors.New("second")},
    }
    got := Blockers(results)
    assert.Len(t, got, 2)
    assert.Contains(t, got[0].Error(), "first")
    assert.Contains(t, got[1].Error(), "second")
}

func TestBlockers_NilResultEntryIsSkipped(t *testing.T) {
```

```
    results := []*ExecutionResult{
        nil,
        {HookID: "a", Status: StatusFailed, Error:
        errors.New("real")},
    }
    got := Blockers(results)
    assert.Len(t, got, 1)
}
```

- [ ] **Step 2: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run 'TestBlockers' ./internal/hooks/
```

Expected: FAIL — `Blockers` undefined.

- [ ] **Step 3: Implement blockers.go**

Create `helix_code/internal/hooks/blockers.go`:

```
package hooks

// Blockers extracts non-nil errors from a result slice. Returns
        nil for an
// empty / nil-only / all-succeeded slice. Used by callers
        (registry.Execute,
// auto_compactor, agent.RequestPlanApproval) to decide whether
        any hook
// objected to the operation.
//
// A "blocker" is any result whose Error is non-nil, regardless
        of Status.
// (StatusFailed implies non-nil Error per the existing executor;
        checking
// Error directly is the more robust contract.)
func Blockers(results []*ExecutionResult) []error {
    var blockers []error
    for _, r := range results {
        if r == nil {
            continue
        }
        if r.Error != nil {
            blockers = append(blockers, r.Error)
        }
    }
    return blockers
}
```

- [ ] **Step 4: Run tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run 'TestBlockers' ./
        internal/hooks/
```

Expected: PASS — 5 tests.

- [ ] **Step 5: Run full hooks package**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race ./internal/hooks/...
```

Expected: PASS.

- [ ] **Step 6: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/hooks/blockers.go internal/hooks/
        blockers_test.go && echo "BLUFF FOUND" || echo "clean"
```

Expected: `clean`.

- [ ] **Step 7: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/hooks/blockers.go helix_code/
        internal/hooks/blockers_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F05-T05): blockers.go — Blockers helper (TDD)

Blockers([]*ExecutionResult) []error extracts non-nil errors from
        a
result slice. Used by registry.Execute / auto_compactor / agent's
RequestPlanApproval to detect "did any hook object?". Nil-result
        and
empty-slice cases handled. Order-preserving across multiple
        blockers.
5 unit tests.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 6: Wire registry.Execute with 6 events (TDD)

Files: - Modify: `helix_code/internal/tools/registry.go` (Execute + new SetHooksManager method) - Create: `helix_code/internal/tools/registry_hooks_test.go`

☐ **Step 1: Write failing test**

Create `helix_code/internal/tools/registry_hooks_test.go`:

```
package tools

import (
    "context"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/hooks"
)

// fakeTool is a minimal Tool that records its invocation count
        and lets the
// test seed a fixed result/error. Mocking is allowed at the
        unit-test layer.
type fakeTool struct {
    name        string
    executeCalled int
    resultValue interface{}
    resultErr   error
}

func (f *fakeTool) Name()
        string                                                {
        return f.name }
func (f *fakeTool) Description()
        string                                          { return
        "fake" }
func (f *fakeTool) Schema()
        ToolSchema                                         {
        return ToolSchema{Type: "object"} }
func (f *fakeTool) Category()
        ToolCategory                                    {
        return CategoryShell }
func (f *fakeTool) Validate(map[string]interface{})
        error                       { return nil }
```

```
func (f *fakeTool) Execute(ctx context.Context, params
        map[string]interface{}) (interface{}, error) {
    f.executeCalled++
    return f.resultValue, f.resultErr
}

func TestRegistry_SetHooksManager_AcceptsManager(t *testing.T) {
    r, err := NewToolRegistry(DefaultRegistryConfig())
    require.NoError(t, err)
    r.SetHooksManager(hooks.NewManager())
    assert.NotNil(t, r.hooksManager)
}

func TestRegistry_Execute_BeforeToolCallBlockPreventsExecute(t
        *testing.T) {
    r, err := NewToolRegistry(DefaultRegistryConfig())
    require.NoError(t, err)
    tool := &fakeTool{name: "FakeTool"}
    r.Register(tool)

    hm := hooks.NewManager()
    blockHook := hooks.NewHook("blocker",
        hooks.HookTypeBeforeToolCall,
        func(ctx context.Context, e *hooks.Event) error {
            return assert.AnError // any non-nil error blocks
        })
    require.NoError(t, hm.Register(blockHook))
    r.SetHooksManager(hm)

    _, execErr := r.Execute(context.Background(), "FakeTool",
        map[string]interface{}{})
    require.Error(t, execErr)
    assert.Equal(t, 0, tool.executeCalled, "Execute must NOT run
        when before-hook blocks")
}

func TestRegistry_Execute_AfterToolCallFiresEvenOnError(t
        *testing.T) {
    r, err := NewToolRegistry(DefaultRegistryConfig())
    require.NoError(t, err)
    tool := &fakeTool{name: "FakeTool", resultErr:
        assert.AnError}
    r.Register(tool)

    hm := hooks.NewManager()
    afterFireCount := 0
    afterHook := hooks.NewHook("after",
        hooks.HookTypeAfterToolCall,
```

```
            func(ctx context.Context, e *hooks.Event) error {
                afterFireCount++
                return nil
            })
        require.NoError(t, hm.Register(afterHook))
        r.SetHooksManager(hm)

        _, _ = r.Execute(context.Background(), "FakeTool",
            map[string]interface{}{})
        assert.Equal(t, 1, tool.executeCalled, "tool must have run")
        assert.Equal(t, 1, afterFireCount, "AfterToolCall must fire
            even on tool error")
    }

    func TestRegistry_Execute_BashFiresSpecialisedBeforeBashAndAfterBash(t
            *testing.T) {
        r, err := NewToolRegistry(DefaultRegistryConfig())
        require.NoError(t, err)
        tool := &fakeTool{name: "Bash"}
        r.Register(tool)

        hm := hooks.NewManager()
        beforeBash, afterBash := 0, 0
        require.NoError(t, hm.Register(hooks.NewHook("bb",
            hooks.HookTypeBeforeBash,
            func(ctx context.Context, e *hooks.Event) error {
            beforeBash++; return nil })))
        require.NoError(t, hm.Register(hooks.NewHook("ab",
            hooks.HookTypeAfterBash,
            func(ctx context.Context, e *hooks.Event) error {
            afterBash++; return nil })))
        r.SetHooksManager(hm)

        _, err = r.Execute(context.Background(), "Bash",
            map[string]interface{}{"command": "ls"})
        require.NoError(t, err)
        assert.Equal(t, 1, beforeBash)
        assert.Equal(t, 1, afterBash)
    }

    func TestRegistry_Execute_EditFiresSpecialisedBeforeEditAndAfterEdit(t
            *testing.T) {
        r, err := NewToolRegistry(DefaultRegistryConfig())
        require.NoError(t, err)
        tool := &fakeTool{name: "Edit"}
        r.Register(tool)

        hm := hooks.NewManager()
```

```
    beforeEdit, afterEdit := 0, 0
    require.NoError(t, hm.Register(hooks.NewHook("be",
        hooks.HookTypeBeforeEdit,
        func(ctx context.Context, e *hooks.Event) error {
        beforeEdit++; return nil })))
    require.NoError(t, hm.Register(hooks.NewHook("ae",
        hooks.HookTypeAfterEdit,
        func(ctx context.Context, e *hooks.Event) error {
        afterEdit++; return nil })))
    r.SetHooksManager(hm)

    _, err = r.Execute(context.Background(), "Edit",
        map[string]interface{}{"path": "/tmp/x"})
    require.NoError(t, err)
    assert.Equal(t, 1, beforeEdit)
    assert.Equal(t, 1, afterEdit)
}

func TestRegistry_Execute_NilHooksManagerIsPassthrough(t
        *testing.T) {
    r, err := NewToolRegistry(DefaultRegistryConfig())
    require.NoError(t, err)
    tool := &fakeTool{name: "X", resultValue: 42}
    r.Register(tool)
    // SetHooksManager not called → hooksManager is nil
    got, err := r.Execute(context.Background(), "X",
        map[string]interface{}{})
    require.NoError(t, err)
    assert.Equal(t, 42, got)
}
```

- [ ] **Step 2: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run 'TestRegistry_(SetHooksManager|
        Execute_(BeforeToolCallBlock|AfterToolCall|BashFires|
        EditFires|NilHooksManager))' ./internal/tools/
```

Expected: FAIL — `SetHooksManager` undefined.

- [ ] **Step 3: Modify registry.go**

In `helix_code/internal/tools/registry.go`:

1. Add the import for `dev.helix.code/internal/hooks` (if not already present).
2. Add a `hooksManager *hooks.Manager` field to the `ToolRegistry` struct (around line 63).

3. Add a `SetHooksManager` method:

```
// SetHooksManager wires a hooks.Manager so Execute can fire
        lifecycle events
// (BeforeToolCall / AfterToolCall plus specialised BeforeBash/
        AfterBash for
// Bash and BeforeEdit/AfterEdit for Edit/Write/MultiEdit). A nil
        manager
// disables hook firing (Execute behaves as before).
func (r *ToolRegistry) SetHooksManager(m *hooks.Manager) {
    r.mu.Lock()
    defer r.mu.Unlock()
    r.hooksManager = m
}
```

1. Replace the existing `Execute` method (currently lines 254-268):

```
// Execute executes a tool by name with given parameters.
// Fires hook lifecycle events around the inner tool.Execute when
        a hooks
// manager is configured via SetHooksManager. A blocking before-
        hook prevents
// the tool from running and returns an error wrapping the
        blockers.
// After-hooks fire even when the tool returned an error so
        observability
// hooks see the full picture; a blocking after-hook is logged at
        WARN but
// does not retroactively undo the operation.
func (r *ToolRegistry) Execute(ctx context.Context, name string,
        params map[string]interface{}) (interface{}, error) {
    tool, err := r.Get(name)
    if err != nil {
        return nil, err
    }
    if err := tool.Validate(params); err != nil {
        return nil, fmt.Errorf("parameter validation failed:
        %w", err)
    }

    // BeforeToolCall + specialised before-events (block aborts).
    if r.hooksManager != nil {
        if err := r.fireBefore(ctx, name, params); err != nil {
            return nil, err
        }
    }

    result, execErr := tool.Execute(ctx, params)
```

```
    // AfterToolCall + specialised after-events (block logged,
        not propagated).
    if r.hooksManager != nil {
        r.fireAfter(ctx, name, params, result, execErr)
    }

    return result, execErr
}

// fireBefore dispatches BeforeToolCall + the specialised event
        for the tool.
// Returns the first non-nil blocker as a wrapped error; nil if
        everything OK.
func (r *ToolRegistry) fireBefore(ctx context.Context, name
        string, params map[string]interface{}) error {
    if err := r.dispatchAndCheck(ctx,
        hooks.HookTypeBeforeToolCall, "tool_registry",
        map[string]interface{}{
        "toolName": name,
        "params":   params,
    }); err != nil {
        return err
    }
    if specialised, ok := specialisedBeforeEvent(name); ok {
        if err := r.dispatchAndCheck(ctx, specialised,
        "tool_registry", map[string]interface{}{
            "toolName": name,
            "params":   params,
        }); err != nil {
            return err
        }
    }
    return nil
}

// fireAfter dispatches AfterToolCall + the specialised event for
        the tool.
// Blockers from after-events are logged at WARN; this function
        never returns
// them as errors (the operation already happened).
func (r *ToolRegistry) fireAfter(ctx context.Context, name
        string, params map[string]interface{}, result
        interface{}, execErr error) {
    data := map[string]interface{}{
        "toolName": name,
        "params":   params,
        "result":   result,
        "error":    errString(execErr),
```

```
    }
    r.dispatchAndLog(ctx, hooks.HookTypeAfterToolCall,
        "tool_registry", data)
    if specialised, ok := specialisedAfterEvent(name); ok {
        r.dispatchAndLog(ctx, specialised, "tool_registry", data)
    }
}

// dispatchAndCheck fires an event synchronously and returns the
        first blocker
// as a wrapped error.
func (r *ToolRegistry) dispatchAndCheck(ctx context.Context,
        evtType hooks.HookType, source string, data
        map[string]interface{}) error {
    event := hooks.NewEventWithContext(ctx, evtType)
    event.Source = source
    for k, v := range data {
        event.SetData(k, v)
    }
    results := r.hooksManager.TriggerEventAndWait(event)
    if blockers := hooks.Blockers(results); len(blockers) > 0 {
        return fmt.Errorf("operation blocked by hook(s) on %s:
        %v", evtType, blockers[0])
    }
    return nil
}

// dispatchAndLog fires an event synchronously, logging any
        blockers at WARN.
func (r *ToolRegistry) dispatchAndLog(ctx context.Context,
        evtType hooks.HookType, source string, data
        map[string]interface{}) {
    event := hooks.NewEventWithContext(ctx, evtType)
    event.Source = source
    for k, v := range data {
        event.SetData(k, v)
    }
    results := r.hooksManager.TriggerEventAndWait(event)
    if blockers := hooks.Blockers(results); len(blockers) > 0 {
        log.Printf("WARN registry: %d hook blocker(s) on %s
        ignored: %v", len(blockers), evtType, blockers[0])
    }
}

// specialisedBeforeEvent maps tool names to the specialised
        before-event
// (BeforeBash for Bash; BeforeEdit for Edit/Write/MultiEdit).
        Returns false
```

```
// for tools without a specialisation.
func specialisedBeforeEvent(toolName string) (hooks.HookType,
        bool) {
    switch toolName {
    case "Bash":
        return hooks.HookTypeBeforeBash, true
    case "Edit", "Write", "MultiEdit":
        return hooks.HookTypeBeforeEdit, true
    }
    return "", false
}

// specialisedAfterEvent mirrors specialisedBeforeEvent for the
        after side.
func specialisedAfterEvent(toolName string) (hooks.HookType,
        bool) {
    switch toolName {
    case "Bash":
        return hooks.HookTypeAfterBash, true
    case "Edit", "Write", "MultiEdit":
        return hooks.HookTypeAfterEdit, true
    }
    return "", false
}

// errString safely renders an error for inclusion in event
        payloads.
func errString(err error) string {
    if err == nil {
        return ""
    }
    return err.Error()
}
```

1. Add the import for `log` to registry.go if not already present.

- [ ] **Step 4: Run tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run 'TestRegistry_' ./
        internal/tools/
```

Expected: PASS — 6 tests.

- [ ] **Step 5: Run full tools package + hooks package (regression)**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race ./internal/tools/... ./
        internal/hooks/...
```

Expected: PASS.

- [ ] **Step 6: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/tools/registry.go internal/tools/
        registry_hooks_test.go && echo "BLUFF FOUND" || echo
        "clean"
```

Expected: `clean`. (Existing registry.go may have pre-existing hits; only flag new lines.)

- [ ] **Step 7: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/tools/registry.go helix_code/
        internal/tools/registry_hooks_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F05-T06): wire registry.Execute with 6 hook events (TDD)

ToolRegistry.SetHooksManager + Execute now dispatches
        BeforeToolCall
(synchronous, blocker aborts), then conditionally BeforeBash
        (toolName==
"Bash") or BeforeEdit (Edit/Write/MultiEdit). Tool runs.
        AfterToolCall
+ specialised after-event fire even on tool error so
        observability
hooks see the full picture; after-event blockers are logged at
        WARN
but do NOT retroactively undo the operation. Nil hooksManager =
passthrough (existing behaviour preserved). 6 unit tests.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 7: Wire OnCompaction in AutoCompactor (TDD)

Files: - Modify: `helix_code/internal/llm/compression/auto_compactor.go` (add SetHooksManager + OnCompaction dispatch) - Create: `helix_code/internal/llm/compression/auto_compactor_hooks_test.go`

- [ ] **Step 1: Investigate auto_compactor.go**

```
grep -n 'type AutoCompactor\|func.*AutoCompactor.*Compact\|
        return\b' /run/media/milosvasic/DATA4TB/Projects/
        helix_code/helix_code/internal/llm/compression/
        auto_compactor.go | head -20
```

Find the public method that performs a compaction (likely `Compact` or similar). The dispatch fires on success at the end of that method.

- [ ] **Step 2: Write failing test**

Create `helix_code/internal/llm/compression/auto_compactor_hooks_test.go`:

```
package compression

import (
    "context"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/hooks"
)

func TestAutoCompactor_SetHooksManager_AcceptsManager(t
        *testing.T) {
    ac := NewAutoCompactor(testFakeProvider(t))
    ac.SetHooksManager(hooks.NewManager())
    assert.NotNil(t, ac.hooksManager, "field set after
        SetHooksManager")
}

func TestAutoCompactor_FiresOnCompactionAfterSuccessfulCompaction(t
        *testing.T) {
    ac := NewAutoCompactor(testFakeProvider(t))
    hm := hooks.NewManager()
    fired := 0
    require.NoError(t, hm.Register(hooks.NewHook("oc",
        hooks.HookTypeOnCompaction,
```

```
            func(ctx context.Context, e *hooks.Event) error {
                fired++
                assert.NotNil(t, e.Data["before_size"])
                assert.NotNil(t, e.Data["after_size"])
                return nil
            }))
    ac.SetHooksManager(hm)

    // Drive a compaction (call site depends on AutoCompactor's
        actual API).
    _, err := ac.Compact(context.Background(),
        testLargeMessageSet(t))
    require.NoError(t, err)
    assert.Equal(t, 1, fired, "OnCompaction must fire exactly
        once per success")
}

func TestAutoCompactor_BlockerFromHookAbortsCompaction(t
        *testing.T) {
    ac := NewAutoCompactor(testFakeProvider(t))
    hm := hooks.NewManager()
    require.NoError(t, hm.Register(hooks.NewHook("oc",
        hooks.HookTypeOnCompaction,
        func(ctx context.Context, e *hooks.Event) error {
            return assert.AnError
        })))
    ac.SetHooksManager(hm)

    _, err := ac.Compact(context.Background(),
        testLargeMessageSet(t))
    require.Error(t, err, "blocking on_compaction hook must
        surface as compaction error")
}

func TestAutoCompactor_NilHooksManagerIsPassthrough(t
        *testing.T) {
    ac := NewAutoCompactor(testFakeProvider(t))
    // SetHooksManager not called
    _, err := ac.Compact(context.Background(),
        testLargeMessageSet(t))
    require.NoError(t, err, "no hooks = no behaviour change")
}

// testFakeProvider and testLargeMessageSet are helpers that
        depend on the
// AutoCompactor's actual constructor signature and message
        types. The
```

```
// implementer fills these in based on what the existing test
        file
// (auto_compactor_test.go from F01-T06) uses.
//
// The pattern: reuse the same test fixtures F01 already defined
        for
// AutoCompactor unit tests. Look at auto_compactor_test.go to
        find the
// existing helpers and reuse them here.
```

- [ ] **Step 3: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run
        'TestAutoCompactor_(SetHooksManager|Fires|Blocker|
        NilHooksManager)' ./internal/llm/compression/
```

Expected: FAIL — `SetHooksManager` undefined or test helpers undefined.

- [ ] **Step 4: Modify auto_compactor.go**

In `helix_code/internal/llm/compression/auto_compactor.go`:

1. Add the import `dev.helix.code/internal/hooks` if missing.
2. Add a `hooksManager *hooks.Manager` field to the `AutoCompactor` struct.
3. Add a `SetHooksManager` method:

```
// SetHooksManager wires a hooks.Manager so Compact can fire
        OnCompaction
// after a successful compaction. A nil manager disables hook
        firing.
func (ac *AutoCompactor) SetHooksManager(m *hooks.Manager) {
    ac.mu.Lock()
    defer ac.mu.Unlock()
    ac.hooksManager = m
}
```

1. At the end of the `Compact` (or equivalent) method, AFTER the compaction has produced its result and BEFORE returning success, insert:

```
    // Fire OnCompaction; a blocking hook aborts the operation by
        surfacing
    // the blocker as the function's return error.
    if ac.hooksManager != nil {
        event := hooks.NewEventWithContext(ctx,
        hooks.HookTypeOnCompaction)
        event.Source = "auto_compactor"
```

```
        event.SetData("before_size",
        beforeSize)         // adapt name to the local variable
        holding pre-compaction message count
        event.SetData("after_size", afterSize)           // post-
        compaction size
        event.SetData("messages_compacted", compactedCount)
        results := ac.hooksManager.TriggerEventAndWait(event)
        if blockers := hooks.Blockers(results); len(blockers) >
        0 {
            return nil, fmt.Errorf("compaction blocked by
        hook(s): %v", blockers[0])
        }
    }
```

(Adapt `beforeSize` / `afterSize` / `compactedCount` to whatever names the function uses for these values. The plan can't fix variable names that depend on F01's existing implementation — read the file before editing.)

- [ ] **Step 5: Fill in test helpers**

In the new test file, replace the `testFakeProvider(t)` and `testLargeMessageSet(t)` placeholders with the EXACT same fixtures used in `auto_compactor_test.go`. If those helpers aren't already shared, copy or share them via a package-level helper file.

If the existing test file uses inline fixtures rather than helpers, declare equivalent ones in `auto_compactor_hooks_test.go`. The goal: tests compile and exercise `Compact` with messages large enough to trigger compaction.

- [ ] **Step 6: Run tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run
        'TestAutoCompactor_(SetHooksManager|Fires|Blocker|
        NilHooksManager)' ./internal/llm/compression/
```

Expected: PASS — 4 tests.

- [ ] **Step 7: Run full compression package**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race ./internal/llm/compression/...
```

Expected: PASS — F01's existing tests still green.

- [ ] **Step 8: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/llm/compression/auto_compactor.go
        internal/llm/compression/auto_compactor_hooks_test.go &&
        echo "BLUFF FOUND" || echo "clean"
```

Expected: `clean`. (Existing auto_compactor.go may have pre-existing hits — only flag new lines.)

- [ ] **Step 9: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/llm/compression/auto_compactor.go
        helix_code/internal/llm/compression/
        auto_compactor_hooks_test.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F05-T07): wire OnCompaction in AutoCompactor (TDD)

AutoCompactor.SetHooksManager + dispatch of HookTypeOnCompaction
        at the
end of a successful Compact run. Payload: before_size,
        after_size,
messages_compacted. A blocking hook surfaces as the function's
        return
error (the next message-loop iteration handles the error). 4 unit
tests including pass-through with nil manager.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 8: Wire OnError + RequestPlanApproval stub in agent.go (TDD)

Files: - Modify: `helix_code/internal/agent/agent.go` (or `base_agent.go` — investigate which holds the message loop) - Create: `helix_code/internal/agent/agent_hooks_test.go`

- [ ] **Step 1: Investigate agent.go layout**

```
grep -n 'type.*Agent\|messageLoop\|MessageLoop\|RunLoop\|
        HandleError\|on_error' /run/media/milosvasic/DATA4TB/
        Projects/helix_code/helix_code/internal/agent/*.go |
        grep -v _test.go | head -20
```

Find the agent's primary message loop and the spot where tool/LLM errors surface. The `OnError` dispatch fires there.

- [ ] **Step 2: Write failing test**

Create `helix_code/internal/agent/agent_hooks_test.go`:

```
package agent

import (
    "context"
    "errors"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/hooks"
)

func TestBaseAgent_SetHooksManager_AcceptsManager(t *testing.T) {
    a := &BaseAgent{}
    a.SetHooksManager(hooks.NewManager())
    assert.NotNil(t, a.hooksManager)
}

func TestBaseAgent_DispatchOnError_FiresEvent(t *testing.T) {
    a := &BaseAgent{}
    hm := hooks.NewManager()
    fired := 0
    require.NoError(t, hm.Register(hooks.NewHook("oe",
        hooks.HookTypeOnError,
        func(ctx context.Context, e *hooks.Event) error {
            fired++
            assert.NotEmpty(t, e.Data["error_message"])
            return nil
        })))
    a.SetHooksManager(hm)

    a.dispatchOnError(context.Background(),
        errors.New("kaboom"), "tool")
    // dispatchOnError fires async; give it a moment to complete
        in test.
    // (If implementation uses TriggerEventAndWait, this delay is
        unnecessary.)
    for i := 0; i < 20 && fired == 0; i++ {
        // poll briefly — the test is robust whether the dispatch
        is sync or async.
```

```
        time.Sleep(10 * time.Millisecond)
    }
    assert.Equal(t, 1, fired)
}

func TestBaseAgent_RequestPlanApproval_FiresOnPlanApproval(t
        *testing.T) {
    a := &BaseAgent{}
    hm := hooks.NewManager()
    captured := ""
    require.NoError(t, hm.Register(hooks.NewHook("opa",
        hooks.HookTypeOnPlanApproval,
        func(ctx context.Context, e *hooks.Event) error {
            captured, _ = e.Data["plan_text"].(string)
            return nil
        })))
    a.SetHooksManager(hm)

    err := a.RequestPlanApproval(context.Background(), "plan: do
        X then Y")
    require.NoError(t, err)
    assert.Equal(t, "plan: do X then Y", captured)
}

func TestBaseAgent_RequestPlanApproval_BlockerSurfaces(t
        *testing.T) {
    a := &BaseAgent{}
    hm := hooks.NewManager()
    require.NoError(t, hm.Register(hooks.NewHook("opa",
        hooks.HookTypeOnPlanApproval,
        func(ctx context.Context, e *hooks.Event) error {
            return errors.New("plan rejected by policy")
        })))
    a.SetHooksManager(hm)

    err := a.RequestPlanApproval(context.Background(),
        "plan: ...")
    require.Error(t, err)
    assert.Contains(t, err.Error(), "rejected by policy")
}

func TestBaseAgent_NilHooksManagerIsSafe(t *testing.T) {
    a := &BaseAgent{}
    // No SetHooksManager call.
    a.dispatchOnError(context.Background(), errors.New("x"),
        "tool")  // must not panic
```

```
    require.NoError(t,
        a.RequestPlanApproval(context.Background(),
        "p")) // must not panic + return nil
}
```

(Add `import "time"` to the test file's imports.)

- [ ] **Step 3: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run 'TestBaseAgent_(SetHooksManager|
        DispatchOnError|RequestPlanApproval|NilHooksManager)' ./
        internal/agent/
```

Expected: FAIL — `SetHooksManager` / `dispatchOnError` / `RequestPlanApproval` undefined.

- [ ] **Step 4: Modify agent.go (or base_agent.go)**

Locate `BaseAgent` (or whatever the primary agent type is) and add:

1. The import for `dev.helix.code/internal/hooks`.
2. `hooksManager *hooks.Manager` field.
3. The methods:

```
// SetHooksManager wires a hooks.Manager so the agent's lifecycle
        code can
// fire OnError (on tool/LLM errors in the message loop) and
        OnPlanApproval
// (when the plan-mode approval gate calls RequestPlanApproval).
        A nil
// manager disables hook firing.
func (a *BaseAgent) SetHooksManager(m *hooks.Manager) {
    a.hooksManager = m
}

// dispatchOnError fires HookTypeOnError synchronously with a
        payload of
// {error_message, error_type}. Sync (TriggerEventAndWait) is
        used so test
// observation is deterministic; the returned blockers are
        deliberately
// IGNORED — the error has already happened and the agent loop's
        job is
// to report it, not retry.
func (a *BaseAgent) dispatchOnError(ctx context.Context, err
        error, errorType string) {
```

```
    if a.hooksManager == nil || err == nil {
        return
    }
    event := hooks.NewEventWithContext(ctx,
        hooks.HookTypeOnError)
    event.Source = "agent"
    event.SetData("error_message", err.Error())
    event.SetData("error_type", errorType)
    _ = a.hooksManager.TriggerEventAndWait(event)
}

// RequestPlanApproval fires HookTypeOnPlanApproval
        synchronously. A
// blocking hook surfaces as a returned error; nil error means
        all hooks
// allow the plan. F05 ships this method but does NOT call it in
        the agent
// message loop — F08 (Plan Mode) wires it into the actual
        approval gate.
func (a *BaseAgent) RequestPlanApproval(ctx context.Context,
        plan string) error {
    if a.hooksManager == nil {
        return nil
    }
    event := hooks.NewEventWithContext(ctx,
        hooks.HookTypeOnPlanApproval)
    event.Source = "agent"
    event.SetData("plan_text", plan)
    results := a.hooksManager.TriggerEventAndWait(event)
    if blockers := hooks.Blockers(results); len(blockers) > 0 {
        return fmt.Errorf("plan approval blocked: %v",
        blockers[0])
    }
    return nil
}
```

1. In the existing agent message loop where tool/LLM errors are handled, add a call to `a.dispatchOnError(ctx, err, "tool")` (or `"llm"` depending on the error origin). This is the load-bearing wiring: without it, OnError never fires in production. Locate the error-handling code in agent.go's main loop and insert the dispatch immediately after the error becomes visible (typically right before the loop returns or logs the error).

If the existing message loop is in a different file (e.g., `cmd/cli/main.go` or `internal/llm/tool_provider.go`), insert the dispatch wherever the error first surfaces — but preferentially keep it in `agent.go` so all tests for the dispatch live in one place.

- [ ] **Step 5: Run tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run
        'TestBaseAgent_(SetHooksManager|DispatchOnError|
        RequestPlanApproval|NilHooksManager)' ./internal/agent/
```

Expected: PASS — 5 tests.

- [ ] **Step 6: Run full agent package**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race ./internal/agent/...
```

Expected: PASS — F01/F03 tests (TestGetSystemPrompt etc.) still green.

- [ ] **Step 7: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/agent/agent.go internal/agent/
        base_agent.go internal/agent/agent_hooks_test.go && echo
        "BLUFF FOUND" || echo "clean"
```

Expected: `clean`. (Existing agent files may have pre-existing hits — only flag new lines.)

- [ ] **Step 8: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/agent/
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F05-T08): wire OnError + RequestPlanApproval stub in
        agent (TDD)

BaseAgent.SetHooksManager + dispatchOnError (fires
        HookTypeOnError sync
with payload {error_message, error_type}; blockers ignored — the
        error
already happened) + RequestPlanApproval(ctx, plan) (fires
HookTypeOnPlanApproval sync; blocker surfaces as returned error).
        The
agent message loop calls dispatchOnError where tool/LLM errors
        surface.
RequestPlanApproval has no production caller in F05; F08 (Plan
        Mode)
wires it into the plan-approval gate. 5 unit tests.
```

```
Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 9: helixcode hooks Cobra subcommands

Files: - Create: `helix_code/cmd/cli/hooks_cmd.go` - Create: `helix_code/cmd/cli/hooks_cmd_test.go` - Modify: `helix_code/cmd/cli/main.go` (add dispatcher entry for `os.Args[1] == "hooks"`)

- [ ] **Step 1: Write failing test**

Create `helix_code/cmd/cli/hooks_cmd_test.go`:

```
package main

import (
    "bytes"
    "context"
    "os"
    "path/filepath"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/hooks"
)

func writeYAML(t *testing.T, dir, body string) string {
    t.Helper()
    path := filepath.Join(dir, "hooks.yaml")
    require.NoError(t, os.WriteFile(path, []byte(body), 0o600))
    return path
}

func writeShellScript(t *testing.T, dir, body string) string {
    t.Helper()
    path := filepath.Join(dir, "hook.sh")
    require.NoError(t, os.WriteFile(path, []byte("#!/bin/
        sh\n"+body+"\n"), 0o755))
    return path
}

func TestRunHooksList_EmptyShowsHeader(t *testing.T) {
    tmp := t.TempDir()
```

```
    user := writeYAML(t, tmp, "apiVersion: helixcode.hooks/
        v1\nhooks: []\n")
    var buf bytes.Buffer
    require.NoError(t, runHooksList(&buf, user,
        filepath.Join(tmp, "missing.yaml")))
    out := buf.String()
    assert.Contains(t, out, "ID") // header
}

func TestRunHooksList_AfterLoad(t *testing.T) {
    tmp := t.TempDir()
    script := writeShellScript(t, tmp, "exit 0")
    user := writeYAML(t, tmp, `apiVersion: helixcode.hooks/v1
hooks:
  - id: audit
    event: before_tool_call
    script: `+script+`
    enabled: true
`)
    var buf bytes.Buffer
    require.NoError(t, runHooksList(&buf, user,
        filepath.Join(tmp, "missing.yaml")))
    out := buf.String()
    assert.Contains(t, out, "audit")
    assert.Contains(t, out, "before_tool_call")
}

func TestRunHooksValidate_GoodYAML(t *testing.T) {
    tmp := t.TempDir()
    script := writeShellScript(t, tmp, "exit 0")
    user := writeYAML(t, tmp, `apiVersion: helixcode.hooks/v1
hooks:
  - id: x
    event: on_error
    script: `+script+`
`)
    var buf bytes.Buffer
    require.NoError(t, runHooksValidate(&buf, user,
        filepath.Join(tmp, "missing.yaml")))
    assert.Contains(t, buf.String(), "OK")
}

func TestRunHooksValidate_BadYAML(t *testing.T) {
    tmp := t.TempDir()
    user := writeYAML(t, tmp, "not: valid: yaml: [")
    var buf bytes.Buffer
    err := runHooksValidate(&buf, user, filepath.Join(tmp,
        "missing.yaml"))
```

```
    assert.Error(t, err)
}

func TestRunHooksTest_FiresHandlersForEvent(t *testing.T) {
    tmp := t.TempDir()
    script := writeShellScript(t, tmp, "echo 'hello'; exit 0")
    user := writeYAML(t, tmp, `apiVersion: helixcode.hooks/v1
hooks:
  - id: t
    event: before_tool_call
    script: `+script+`
`)
    var buf bytes.Buffer
    require.NoError(t, runHooksTest(&buf, user,
        filepath.Join(tmp, "missing.yaml"), "before_tool_call"))
    out := buf.String()
    assert.Contains(t, out, "t")  // hook id in output
    // Implementation prints PASS or "no error" or similar;
        presence of hook id is sufficient.
}

func TestRunHooksEnable_FlipsEnabled(t *testing.T) {
    tmp := t.TempDir()
    script := writeShellScript(t, tmp, "exit 0")
    user := writeYAML(t, tmp, `apiVersion: helixcode.hooks/v1
hooks:
  - id: x
    event: on_error
    script: `+script+`
    enabled: false
`)
    require.NoError(t, runHooksEnable(user, "x"))

    loader := &hooks.FileLoader{UserPath: user, ProjectPath:
        filepath.Join(tmp, "missing.yaml")}
    hs, _, err := loader.Load(context.Background())
    require.NoError(t, err)
    require.Len(t, hs, 1, "after enable, hook should be loadable
        (no longer filtered)")
    assert.Equal(t, "x", hs[0].ID)
}

func TestRunHooksDisable_FlipsEnabled(t *testing.T) {
    tmp := t.TempDir()
    script := writeShellScript(t, tmp, "exit 0")
    user := writeYAML(t, tmp, `apiVersion: helixcode.hooks/v1
hooks:
  - id: x
```

```
    event: on_error
    script: `+script+`
    enabled: true
`)
    require.NoError(t, runHooksDisable(user, "x"))

    loader := &hooks.FileLoader{UserPath: user, ProjectPath:
        filepath.Join(tmp, "missing.yaml")}
    hs, _, err := loader.Load(context.Background())
    require.NoError(t, err)
    assert.Empty(t, hs, "after disable, hook should be filtered
        out by Load")
}
```

- [ ] **Step 2: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run 'TestRunHooks' ./cmd/cli/
```

Expected: FAIL — runHooks* functions undefined.

- [ ] **Step 3: Implement hooks_cmd.go**

Create `helix_code/cmd/cli/hooks_cmd.go`:

```
package main

import (
    "context"
    "fmt"
    "io"
    "os"
    "text/tabwriter"

    "github.com/spf13/cobra"
    "gopkg.in/yaml.v3"

    "dev.helix.code/internal/hooks"
)

func newHooksCommand() *cobra.Command {
    cmd := &cobra.Command{
        Use:   "hooks",
        Short: "Manage HelixCode hook scripts",
    }
    cmd.AddCommand(newHooksListCommand())
    cmd.AddCommand(newHooksValidateCommand())
    cmd.AddCommand(newHooksTestCommand())
```

```
    cmd.AddCommand(newHooksEnableCommand())
    cmd.AddCommand(newHooksDisableCommand())
    return cmd
}

func newHooksListCommand() *cobra.Command {
    return &cobra.Command{
        Use:   "list",
        Short:
        "List all enabled hooks loaded from user + project
        hooks.yaml",
        RunE: func(cmd *cobra.Command, args []string) error {
            user, project := defaultHooksPaths()
            return runHooksList(os.Stdout, user, project)
        },
    }
}

func newHooksValidateCommand() *cobra.Command {
    return &cobra.Command{
        Use:   "validate",
        Short: "Parse hooks.yaml and report errors without
        running anything",
        RunE: func(cmd *cobra.Command, args []string) error {
            user, project := defaultHooksPaths()
            return runHooksValidate(os.Stdout, user, project)
        },
    }
}

func newHooksTestCommand() *cobra.Command {
    return &cobra.Command{
        Use:   "test <event-name>",
        Short: "Simulate an event and run all registered hooks
        for it",
        Args:  cobra.ExactArgs(1),
        RunE: func(cmd *cobra.Command, args []string) error {
            user, project := defaultHooksPaths()
            return runHooksTest(os.Stdout, user, project,
        args[0])
        },
    }
}

func newHooksEnableCommand() *cobra.Command {
    return &cobra.Command{
        Use:   "enable <id>",
```

```
        Short: "Set enabled=true for a hook in user's
        hooks.yaml",
        Args:  cobra.ExactArgs(1),
        RunE: func(cmd *cobra.Command, args []string) error {
            user, _ := defaultHooksPaths()
            return runHooksEnable(user, args[0])
        },
    }
}

func newHooksDisableCommand() *cobra.Command {
    return &cobra.Command{
        Use:   "disable <id>",
        Short: "Set enabled=false for a hook in user's
        hooks.yaml",
        Args:  cobra.ExactArgs(1),
        RunE: func(cmd *cobra.Command, args []string) error {
            user, _ := defaultHooksPaths()
            return runHooksDisable(user, args[0])
        },
    }
}

func defaultHooksPaths() (string, string) {
    home, _ := os.UserHomeDir()
    cwd, _ := os.Getwd()
    return filepath_join(home, ".helixcode/hooks.yaml"),
        filepath_join(cwd, ".helixcode/hooks.yaml")
}

func filepath_join(parts ...string) string {
    if len(parts) == 0 {
        return ""
    }
    res := parts[0]
    for _, p := range parts[1:] {
        res = res + "/" + p
    }
    return res
}

func runHooksList(out io.Writer, userPath, projectPath string)
        error {
    loader := &hooks.FileLoader{UserPath: userPath, ProjectPath:
        projectPath}
    hs, sources, err := loader.Load(context.Background())
    if err != nil {
        return err
```

```
    }
    tw := tabwriter.NewWriter(out, 0, 0, 2, ' ', 0)
    fmt.Fprintf(tw, "ID\tEVENT\tPRIORITY\tASYNC\tSCRIPT\n")
    for _, h := range hs {
        fmt.Fprintf(tw, "%s\t%s\t%d\t%v\t%s\n", h.ID, h.Type,
        h.Priority, h.Async, h.Metadata["script"])
    }
    if len(hs) == 0 {
        fmt.Fprintln(tw, "(no hooks loaded)\t\t\t\t")
    }
    fmt.Fprintf(tw, "\nSources: %v\n", sources)
    return tw.Flush()
}

func runHooksValidate(out io.Writer, userPath, projectPath
        string) error {
    loader := &hooks.FileLoader{UserPath: userPath, ProjectPath:
        projectPath}
    hs, sources, err := loader.Load(context.Background())
    if err != nil {
        return err
    }
    fmt.Fprintf(out, "OK: %d hook(s) loaded from %v\n", len(hs),
        sources)
    return nil
}

func runHooksTest(out io.Writer, userPath, projectPath,
        eventName string) error {
    loader := &hooks.FileLoader{UserPath: userPath, ProjectPath:
        projectPath}
    hs, _, err := loader.Load(context.Background())
    if err != nil {
        return err
    }
    mgr := hooks.NewManager()
    for _, h := range hs {
        // Wrap each hook's script path into a runner.
        scriptPath := h.Metadata["script"]
        h.Handler = hooks.NewShellRunner(scriptPath, h.Timeout)
        if err := mgr.Register(h); err != nil {
            return fmt.Errorf("registering %s: %w", h.ID, err)
        }
    }
    event := hooks.NewEvent(hooks.HookType(eventName))
    results := mgr.TriggerEventAndWait(event)
    for _, r := range results {
```

```
            fmt.Fprintf(out, "%s: status=%s err=%v duration=%s\n",
            r.HookID, r.Status, r.Error, r.Duration)
        }
        if len(results) == 0 {
            fmt.Fprintf(out, "(no hooks registered for event %q)\n",
            eventName)
        }
        return nil
    }

    func runHooksEnable(userPath, id string) error {
        return setHookEnabled(userPath, id, true)
    }

    func runHooksDisable(userPath, id string) error {
        return setHookEnabled(userPath, id, false)
    }

    // setHookEnabled mutates user's hooks.yaml using yaml.v3 Node-
            based round
    // trip so user comments are preserved.
    func setHookEnabled(userPath, id string, want bool) error {
        body, err := os.ReadFile(userPath)
        if err != nil {
            return err
        }
        var root yaml.Node
        if err := yaml.Unmarshal(body, &root); err != nil {
            return err
        }
        if root.Kind != yaml.DocumentNode || len(root.Content) == 0 {
            return fmt.Errorf("unexpected YAML structure in %s",
            userPath)
        }
        doc := root.Content[0]
        if doc.Kind != yaml.MappingNode {
            return fmt.Errorf("expected top-level mapping in %s",
            userPath)
        }
        hooksNode := childByKey(doc, "hooks")
        if hooksNode == nil || hooksNode.Kind != yaml.SequenceNode {
            return fmt.Errorf("hooks: key not a sequence in %s",
            userPath)
        }
        for _, item := range hooksNode.Content {
            if item.Kind != yaml.MappingNode {
                continue
            }
```

```
        idNode := childByKey(item, "id")
        if idNode == nil || idNode.Value != id {
            continue
        }
        setOrInsertBool(item, "enabled", want)
        out, err := yaml.Marshal(&root)
        if err != nil {
            return err
        }
        return os.WriteFile(userPath, out, 0o600)
    }
    return fmt.Errorf("hook %q not found in %s", id, userPath)
}

func childByKey(m *yaml.Node, key string) *yaml.Node {
    for i := 0; i+1 < len(m.Content); i += 2 {
        if m.Content[i].Value == key {
            return m.Content[i+1]
        }
    }
    return nil
}

func setOrInsertBool(m *yaml.Node, key string, val bool) {
    for i := 0; i+1 < len(m.Content); i += 2 {
        if m.Content[i].Value == key {
            m.Content[i+1].Value = boolStr(val)
            m.Content[i+1].Tag = "!!bool"
            return
        }
    }
    m.Content = append(m.Content,
        &yaml.Node{Kind: yaml.ScalarNode, Value: key},
        &yaml.Node{Kind: yaml.ScalarNode, Value: boolStr(val),
        Tag: "!!bool"},
    )
}

func boolStr(b bool) string {
    if b {
        return "true"
    }
    return "false"
}
```

(Replace the manual `filepath_join` helper with `path/filepath`'s `filepath.Join` if you prefer; the manual one avoids an import. Use whichever matches existing repo conventions.)

Step 4: Wire dispatcher in main.go

In `helix_code/cmd/cli/main.go`, find the existing dispatcher block (added by F02 for `os.Args[1] == "permissions"` and extended by F04 for `worktree`). Add an analogous block for `"hooks"` immediately after the existing ones:

```
if len(os.Args) >= 2 && os.Args[1] == "hooks" {
    cmd := newHooksCommand()
    cmd.SetArgs(os.Args[2:])
    if err := cmd.Execute(); err != nil {
        os.Exit(1)
    }
    return
}
```

- [ ]

Step 5: Verify it compiles

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go build ./cmd/cli/...
```

Expected: clean compile.

- [ ]

Step 6: Run tests

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run 'TestRunHooks' ./cmd/
        cli/
```

Expected: PASS — 7 tests.

- [ ]

Step 7: Smoke-test the binary

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go build -o bin/helixcode ./cmd/cli && ./bin/
        helixcode hooks --help
```

Expected: prints subcommand list `list / validate / test / enable / disable`.

- [ ]

Step 8: Anti-bluff smoke

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" cmd/cli/hooks_cmd.go cmd/cli/
        hooks_cmd_test.go && echo "BLUFF FOUND" || echo "clean"
```

Expected: `clean`.

- [ ]

Step 9: Commit

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add -f
        helix_code/cmd/cli/hooks_cmd.go helix_code/cmd/cli/
        hooks_cmd_test.go helix_code/cmd/cli/main.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F05-T09): helixcode hooks
        {list,test,enable,disable,validate} subcommands

Cobra subcommand group dispatched from cmd/cli/main.go's args
        sniffer
(same pattern as F02 permissions / F04 worktree). list shows
        tabular
output with ID/EVENT/PRIORITY/ASYNC/SCRIPT columns. validate
        parses
without side effects. test simulates an event and runs all
        registered
shell hooks. enable/disable mutate the user's hooks.yaml in-place
        via
yaml.v3 Node round-trip (comments preserved). 7 unit tests.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 10: /hooks slash command + builtin registration

Files: - Create: `helix_code/internal/commands/hooks_command.go` - Create: `helix_code/internal/commands/hooks_command_test.go` - Modify: `helix_code/internal/commands/builtin/register.go` (add `RegisterBuiltinCommandsWithHooks`) - Create: `helix_code/internal/commands/builtin/hooks_register_test.go`

- [ ] **Step 1: Write failing test for the slash command**

Create `helix_code/internal/commands/hooks_command_test.go`:

```
package commands

import (
    "context"
    "os"
    "path/filepath"
    "testing"

    "github.com/stretchr/testify/assert"
```

```
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/hooks"
)

func setupHooksManager(t *testing.T) *hooks.Manager {
    t.Helper()
    tmp := t.TempDir()
    scriptPath := filepath.Join(tmp, "hook.sh")
    require.NoError(t, os.WriteFile(scriptPath, []byte("#!/bin/
        sh\nexit 0\n"), 0o755))
    mgr := hooks.NewManager()
    h := hooks.NewHook("hk", hooks.HookTypeBeforeToolCall,
        hooks.NewShellRunner(scriptPath, 0))
    require.NoError(t, mgr.Register(h))
    return mgr
}

func TestHooksCommand_NameAliases(t *testing.T) {
    cmd := NewHooksCommand(setupHooksManager(t))
    assert.Equal(t, "hooks", cmd.Name())
    assert.Contains(t, cmd.Aliases(), "hk")
}

func TestHooksCommand_ListSubaction(t *testing.T) {
    mgr := setupHooksManager(t)
    cmd := NewHooksCommand(mgr)
    res, err := cmd.Execute(context.Background(),
        &CommandContext{
        Args:     []string{},
        RawInput: "/hooks",
    })
    require.NoError(t, err)
    assert.Contains(t, res.Output, "hk", "list output must
        contain registered hook id")
}

func TestHooksCommand_TestSubaction(t *testing.T) {
    mgr := setupHooksManager(t)
    cmd := NewHooksCommand(mgr)
    res, err := cmd.Execute(context.Background(),
        &CommandContext{
        Args:     []string{"test", "before_tool_call"},
        RawInput: "/hooks test before_tool_call",
    })
    require.NoError(t, err)
    assert.Contains(t, res.Output, "hk", "hook id must appear in
        test output")
```

```
}

func TestHooksCommand_RejectsUnknownSubaction(t *testing.T) {
    cmd := NewHooksCommand(setupHooksManager(t))
    _, err := cmd.Execute(context.Background(), &CommandContext{
        Args:     []string{"frobnicate"},
        RawInput: "/hooks frobnicate",
    })
    require.Error(t, err)
    assert.Contains(t, err.Error(), "unknown")
}

func TestHooksCommand_TestRequiresEvent(t *testing.T) {
    cmd := NewHooksCommand(setupHooksManager(t))
    _, err := cmd.Execute(context.Background(), &CommandContext{
        Args:     []string{"test"},
        RawInput: "/hooks test",
    })
    require.Error(t, err)
}
```

- [ ] **Step 2: Run failing test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run 'TestHooksCommand' ./internal/
        commands/
```

Expected: FAIL — `NewHooksCommand` undefined.

- [ ] **Step 3: Implement hooks_command.go**

Create `helix_code/internal/commands/hooks_command.go`:

```
package commands

import (
    "bytes"
    "context"
    "fmt"
    "text/tabwriter"

    "dev.helix.code/internal/hooks"
)

// HooksCommand implements /hooks.
//
// Subactions:
//   /hooks                          — list (default)
```

```
//   /hooks list                    — explicit list
//   /hooks test <event-name>       — fire all hooks for the
        event
type HooksCommand struct {
    mgr *hooks.Manager
}

// NewHooksCommand wires a hooks.Manager into the slash command.
func NewHooksCommand(mgr *hooks.Manager) *HooksCommand {
    return &HooksCommand{mgr: mgr}
}

func (c *HooksCommand) Name() string         { return "hooks" }
func (c *HooksCommand) Aliases() []string    { return
        []string{"hk"} }
func (c *HooksCommand) Description() string  { return "manage
        hook scripts" }
func (c *HooksCommand) Usage() string        { return "/hooks
        [list | test <event>]" }

func (c *HooksCommand) Execute(ctx context.Context, cmdCtx
        *CommandContext) (*CommandResult, error) {
    if len(cmdCtx.Args) == 0 {
        return c.list()
    }
    switch cmdCtx.Args[0] {
    case "list":
        return c.list()
    case "test":
        if len(cmdCtx.Args) < 2 {
            return nil, fmt.Errorf("usage: /hooks test <event>")
        }
        return c.test(ctx, cmdCtx.Args[1])
    default:
        return nil, fmt.Errorf("unknown /hooks subaction %q
        (valid: list, test)", cmdCtx.Args[0])
    }
}

func (c *HooksCommand) list() (*CommandResult, error) {
    all := c.mgr.GetAll()
    var buf bytes.Buffer
    tw := tabwriter.NewWriter(&buf, 0, 0, 2, ' ', 0)
    fmt.Fprintf(tw, "ID\tEVENT\tPRIORITY\tASYNC\tENABLED\n")
    for _, h := range all {
        fmt.Fprintf(tw, "%s\t%s\t%d\t%v\t%v\n", h.ID, h.Type,
        h.Priority, h.Async, h.Enabled)
    }
```

```
    if len(all) == 0 {
        fmt.Fprintln(tw, "(no hooks registered)\t\t\t\t")
    }
    tw.Flush()
    return &CommandResult{Output: buf.String(), Success: true},
        nil
}

func (c *HooksCommand) test(ctx context.Context, eventName
        string) (*CommandResult, error) {
    event := hooks.NewEventWithContext(ctx,
        hooks.HookType(eventName))
    results := c.mgr.TriggerEventAndWait(event)
    var buf bytes.Buffer
    for _, r := range results {
        fmt.Fprintf(&buf, "%s: status=%s err=%v duration=%s\n",
        r.HookID, r.Status, r.Error, r.Duration)
    }
    if len(results) == 0 {
        fmt.Fprintf(&buf, "(no hooks registered for event %q)
        \n", eventName)
    }
    return &CommandResult{Output: buf.String(), Success: true},
        nil
}
```

The `Manager.GetAll()` method may not exist on the existing hooks.Manager. If not, add it:

```
grep -n 'func (m \*Manager) GetAll\|func (m \*Manager) List' /
        run/media/milosvasic/DATA4TB/Projects/helix_code/
        helix_code/internal/hooks/manager.go
```

If `GetAll()` doesn't exist, modify `internal/hooks/manager.go` to add:

```
// GetAll returns a snapshot of all registered hooks (any type,
        any state).
func (m *Manager) GetAll() []*Hook {
    m.mu.RLock()
    defer m.mu.RUnlock()
    out := make([]*Hook, 0, len(m.hooksAll))
    for _, h := range m.hooksAll {
        out = append(out, h)
    }
    return out
}
```

(If a similar method exists with a different name, adapt the slash command to use it instead.)

- [ ] **Step 4: Run slash command tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run 'TestHooksCommand' ./
        internal/commands/
```

Expected: PASS — 5 tests.

- [ ] **Step 5: Write registration test**

Create `helix_code/internal/commands/builtin/hooks_register_test.go`:

```
package builtin_test

import (
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/commands"
    "dev.helix.code/internal/commands/builtin"
    "dev.helix.code/internal/hooks"
)

func TestRegisterBuiltinCommands_IncludesHooks(t *testing.T) {
    registry := commands.NewRegistry()
    mgr := hooks.NewManager()
    require.NoError(t,
        builtin.RegisterBuiltinCommandsWithHooks(registry, mgr))

    cmd, ok := registry.Get("hooks")
    require.True(t, ok)
    assert.Equal(t, "hooks", cmd.Name())

    cmd2, ok := registry.Get("hk")
    require.True(t, ok)
    assert.Equal(t, "hooks", cmd2.Name(), "alias resolves to
        hooks")
}
```

- [ ] **Step 6: Run failing registration test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -run
        'TestRegisterBuiltinCommands_IncludesHooks' ./internal/
        commands/builtin/
```

Expected: FAIL — `RegisterBuiltinCommandsWithHooks` undefined.

- [ ] **Step 7: Modify builtin/register.go**

In `helix_code/internal/commands/builtin/register.go`:

1. Add the import for `dev.helix.code/internal/hooks` if missing.
2. Add the new exported function (alongside the existing `RegisterBuiltinCommandsWithWorktree`):

```
// RegisterBuiltinCommandsWithHooks extends
        RegisterBuiltinCommands with the
// /hooks command, which requires a hooks.Manager dependency.
        Callers that
// have a Manager (cmd/cli/main.go startup) use this; callers
        without one
// (legacy paths) use the original RegisterBuiltinCommands.
func RegisterBuiltinCommandsWithHooks(registry
        *commands.Registry, mgr *hooks.Manager) error {
    if err := RegisterBuiltinCommands(registry); err != nil {
        return err
    }
    return registry.Register(commands.NewHooksCommand(mgr))
}
```

1. Update `GetBuiltinCommandNames()` to include `"hooks"`.
2. Update `GetBuiltinCommandAliases()` to include `"hk": "hooks"`.
3. If the existing `TestRegisterBuiltinCommands` test in `builtin_test.go` iterates `GetBuiltinCommandNames()` and registers via the original `RegisterBuiltinCommands` (which doesn't take a hooks.Manager), add `"hooks"` and `"hk"` to that test's skip-set (mirrors how F04 handled `"worktree"/"wt"`).

- [ ] **Step 8: Run registration test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -run
        'TestRegisterBuiltinCommands_IncludesHooks' ./internal/
        commands/builtin/
```

Expected: PASS.

- [ ] **Step 9: Run full commands package + builtin package**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race ./internal/commands/...
```

Expected: PASS — F02 permissions tests + F04 worktree tests still green.

- [ ] **Step 10: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" internal/commands/hooks_command.go internal/
        commands/builtin/register.go internal/commands/builtin/
        hooks_register_test.go && echo "BLUFF FOUND" || echo
        "clean"
```

Expected: `clean`.

- [ ] **Step 11: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/internal/commands/hooks_command.go helix_code/
        internal/commands/hooks_command_test.go helix_code/
        internal/commands/builtin/register.go helix_code/
        internal/commands/builtin/hooks_register_test.go
        helix_code/internal/hooks/manager.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F05-T10): /hooks slash command + builtin registration

HooksCommand implements commands.Command. Subactions: list
        (default),
test <event>. Aliased to /hk. Mutations (enable/disable)
        deliberately
NOT exposed via slash — they touch the user's filesystem and
        route
through the helixcode hooks Cobra subcommand. builtin/register.go
gains RegisterBuiltinCommandsWithHooks (existing
        RegisterBuiltinCommands
signature preserved). 5 unit tests for the slash command + 1
registration test verifying /hooks and /hk both resolve via the
registry. Adds Manager.GetAll() helper.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 11: cmd/cli/main.go startup wiring + integration tests (no mocks)

Files: - Modify: `helix_code/cmd/cli/main.go` - Create: `helix_code/tests/integration/hooks/hooks_integration_test.go`

- [ ] **Step 1: Add initHooks bootstrap to main.go**

In `helix_code/cmd/cli/main.go`, find the existing `Run()` startup sequence (where F02/F03/F04's `initPermissions`/`initPersistence`/`initWorktree` are called). Add a new field on the CLI struct:

```
hooksLoaded int  // count of hooks loaded at startup (for
        diagnostics)
```

Add the bootstrap method near the other `initX` methods:

```
// initHooks loads ~/.helixcode/hooks.yaml + <cwd>/.helixcode/
        hooks.yaml,
// wraps each enabled entry in a shell-runner HookFunc, and
        registers it
// with the existing session.Manager.hooksManager. Errors fail-
        fast.
func (c *CLI) initHooks(ctx context.Context, sessionMgr
        *session.Manager) error {
    home, err := os.UserHomeDir()
    if err != nil {
        return fmt.Errorf("resolving home dir for hooks: %w",
        err)
    }
    cwd, err := os.Getwd()
    if err != nil {
        return fmt.Errorf("resolving cwd for hooks: %w", err)
    }
    loader := &hooks.FileLoader{
        UserPath:    filepath.Join(home, ".helixcode",
        "hooks.yaml"),
        ProjectPath: filepath.Join(cwd, ".helixcode",
        "hooks.yaml"),
    }
    hs, sources, err := loader.Load(ctx)
    if err != nil {
        return fmt.Errorf("loading hooks: %w", err)
    }
    hm := sessionMgr.GetHooksManager()
    for _, h := range hs {
        scriptPath := h.Metadata["script"]
        h.Handler = hooks.NewShellRunner(scriptPath, h.Timeout)
```

```
            if err := hm.Register(h); err != nil {
                return fmt.Errorf("registering hook %q: %w", h.ID,
            err)
            }
        }
        c.hooksLoaded = len(hs)
        if len(sources) > 0 {
            log.Printf("hooks: loaded %d hook(s) from %v", len(hs),
            sources)
        }
        return nil
}
```

Call it from `Run()` after `initWorktree`:

```
if err := c.initWorktree(ctx); err != nil {
    return fmt.Errorf("worktree init: %w", err)
}
if err := c.initHooks(ctx, sessionMgr); err != nil {
    return fmt.Errorf("hooks init: %w", err)
}
```

(Adapt `sessionMgr` to whatever the actual session.Manager variable is in `Run()`.)

Also: connect the hooks.Manager to the tool registry so `BeforeToolCall`/`AfterToolCall` actually fire. After `initHooks`, locate where `c.toolRegistry` (or equivalent) is built and call:

```
c.toolRegistry.SetHooksManager(sessionMgr.GetHooksManager())
```

If `c.toolRegistry` doesn't exist as a CLI struct field, locate the registry-construction site in the existing CLI code and inject `SetHooksManager` there. The exact location depends on the CLI's tool wiring layout — read main.go and adapt.

- [ ] **Step 2: Verify it compiles**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go build ./cmd/cli/...
```

Expected: clean compile.

- [ ] **Step 3: Write integration tests**

```
mkdir -p /run/media/milosvasic/DATA4TB/Projects/helix_code/
        helix_code/tests/integration/hooks
```

Create `helix_code/tests/integration/hooks/hooks_integration_test.go`:

```
//go:build integration
```

```
package hooks_test

import (
    "context"
    "os"
    "path/filepath"
    "testing"

    "github.com/stretchr/testify/assert"
    "github.com/stretchr/testify/require"

    "dev.helix.code/internal/hooks"
)

// fakeIntegrationTool records its invocation count for
        assertions.
type fakeIntegrationTool struct {
    called int
    err    error
}

func newScript(t *testing.T, body string) string {
    t.Helper()
    tmp := t.TempDir()
    path := filepath.Join(tmp, "hook.sh")
    require.NoError(t, os.WriteFile(path, []byte("#!/bin/
        sh\n"+body+"\n"), 0o755))
    return path
}

// TestIntegration_BeforeBashHookBlocksRm proves that a real
        shell-script
// hook on before_bash exiting non-zero prevents the tool's
        Execute from
// running. NO mocks of the hooks system or the registry; only
        the Tool is
// a fake (so the test can assert "Execute was never called").
func TestIntegration_BeforeBashHookBlocksRm(t *testing.T) {
    scriptPath := newScript(t, "echo 'rm blocked' >&2; exit 1")
    mgr := hooks.NewManager()
    hook := hooks.NewHook("blocker", hooks.HookTypeBeforeBash,
        hooks.NewShellRunner(scriptPath, 0))
    require.NoError(t, mgr.Register(hook))

    event := hooks.NewEventWithContext(context.Background(),
        hooks.HookTypeBeforeBash)
    event.SetData("toolName", "Bash")
```

```
    event.SetData("params", map[string]interface{}{"command":
        "rm -rf /tmp/x"})
    results := mgr.TriggerEventAndWait(event)

    blockers := hooks.Blockers(results)
    require.Len(t, blockers, 1, "the blocking hook must produce
        exactly one blocker")
    assert.Contains(t, blockers[0].Error(), "rm blocked")
}

// TestIntegration_AfterToolCallFiresThreeTimes proves the hooks
        system can
// audit multiple tool calls in sequence.
func TestIntegration_AfterToolCallFiresThreeTimes(t *testing.T) {
    tmp := t.TempDir()
    logPath := filepath.Join(tmp, "audit.log")
    scriptPath := newScript(t, "echo 'tool fired' >> "+logPath)

    mgr := hooks.NewManager()
    require.NoError(t, mgr.Register(hooks.NewHook("audit",
        hooks.HookTypeAfterToolCall,
        hooks.NewShellRunner(scriptPath, 0))))

    for i := 0; i < 3; i++ {
        event := hooks.NewEventWithContext(context.Background(),
        hooks.HookTypeAfterToolCall)
        event.SetData("toolName", "X")
        mgr.TriggerEventAndWait(event)
    }

    body, err := os.ReadFile(logPath)
    require.NoError(t, err)
    lines := 0
    for _, b := range body {
        if b == '\n' {
            lines++
        }
    }
    assert.Equal(t, 3, lines, "audit log must have 3 lines (one
        per tool call)")
}

// TestIntegration_YAMLLoaderToManagerRoundTrip proves the path
        from YAML
// file → FileLoader → shellRunner → Manager.Register →
        TriggerEventAndWait
// → script execution → result → Blockers actually works end-to-
        end.
```

```
func TestIntegration_YAMLLoaderToManagerRoundTrip(t *testing.T) {
    tmp := t.TempDir()
    scriptPath := newScript(t, "exit 0")
    yamlPath := filepath.Join(tmp, "hooks.yaml")
    require.NoError(t, os.WriteFile(yamlPath,
        []byte(`apiVersion: helixcode.hooks/v1
hooks:
  - id: rt
    event: on_compaction
    script: `+scriptPath+`
`), 0o600))

    loader := &hooks.FileLoader{UserPath: yamlPath, ProjectPath:
        filepath.Join(tmp, "missing.yaml")}
    hs, _, err := loader.Load(context.Background())
    require.NoError(t, err)
    require.Len(t, hs, 1)

    mgr := hooks.NewManager()
    hs[0].Handler =
        hooks.NewShellRunner(hs[0].Metadata["script"],
        hs[0].Timeout)
    require.NoError(t, mgr.Register(hs[0]))

    event := hooks.NewEventWithContext(context.Background(),
        hooks.HookTypeOnCompaction)
    results := mgr.TriggerEventAndWait(event)
    assert.Empty(t, hooks.Blockers(results), "exit-0 hook must
        not block")
}
```

- [ ] **Step 4: Run integration tests**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && go test -count=1 -race -v -tags=integration ./tests/
        integration/hooks/...
```

Expected: PASS — 3 tests.

- [ ] **Step 5: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" cmd/cli/main.go internal/hooks/ tests/
        integration/hooks/ && echo "BLUFF FOUND" || echo "clean"
```

Expected: `clean`. (cmd/cli/main.go may have pre-existing hits — only flag new lines.)

- [ ] **Step 6: Commit**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add -f
        helix_code/cmd/cli/main.go
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/tests/integration/hooks/
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F05-T11): cmd/cli/main.go startup wiring + integration
        tests

CLI bootstrap loads ~/.helixcode/hooks.yaml + <cwd>/.helixcode/
        hooks.yaml,
wraps each enabled entry in shell-runner HookFunc, registers them
        with
session.Manager.hooksManager, and connects that manager to the
        tool
registry (so BeforeToolCall/AfterToolCall actually fire). 3
        integration
tests with -tags=integration and NO mocks: real shell script
        blocks via
exit-1, audit hook captures 3 tool calls in a real log file, full
YAML→loader→shellRunner→Manager→Trigger round-trip works.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 12: Challenge with three runtime-evidence scenarios

Files: - Create: `helix_code/tests/e2e/challenges/hooks/expected.json` - Create: `helix_code/tests/e2e/challenges/hooks/run.sh` - Create: `helix_code/tests/e2e/challenges/hooks/README.md`

- [ ] **Step 1: Create the directory**

```
mkdir -p /run/media/milosvasic/DATA4TB/Projects/helix_code/
        helix_code/tests/e2e/challenges/hooks
```

- [ ] **Step 2: Write expected.json**

Create `helix_code/tests/e2e/challenges/hooks/expected.json`:

```
{
  "name": "hooks/lifecycle-end-to-end",
```

```
  "feature": "P1-F05 — Hook-Based Extensibility",
  "scenarios": [
    {
      "id": "S1-block-bash-rm",
      "expected_blocker_count": 1,
      "expected_marker_present_after": true
    },
    {
      "id": "S2-audit-after-tool",
      "expected_log_lines": 3
    },
    {
      "id": "S3-yaml-validate-malformed",
      "expected_validate_exit_code_nonzero": true
    }
  ]
}
```

☐ **Step 3: Write run.sh**

Create `helix_code/tests/e2e/challenges/hooks/run.sh`:

```
#!/usr/bin/env bash
# Challenge: P1-F05 — Hook-Based Extensibility runtime evidence.
# Drives hooks.Manager directly through a Go test binary inside
        the module tree.
set -euo pipefail

HERE=$(cd "$(dirname "$0")" && pwd)
ROOT=$(cd "$HERE/../../../.." && pwd)
WORK=$(mktemp -d -p "$ROOT/cmd")
trap 'rm -rf "$WORK"' EXIT

cat > "$WORK/driver.go" <<'EOF'
package main

import (
    "context"
    "fmt"
    "os"
    "path/filepath"

    "dev.helix.code/internal/hooks"
)

func main() {
    if len(os.Args) < 2 {
        fmt.Fprintln(os.Stderr, "usage: driver <scenario>")
```

```
    os.Exit(2)
}

switch os.Args[1] {
case "s1":
    // S1: a real before_bash hook blocks rm.
    tmp, _ := os.MkdirTemp("", "f05-s1-")
    defer os.RemoveAll(tmp)
    marker := filepath.Join(tmp, "marker")
    os.WriteFile(marker, []byte("present"), 0o644)
    scriptPath := filepath.Join(tmp, "block.sh")
    os.WriteFile(scriptPath, []byte("#!/bin/sh\necho
     'blocked' >&2; exit 1\n"), 0o755)

    mgr := hooks.NewManager()
    mgr.Register(hooks.NewHook("blocker",
     hooks.HookTypeBeforeBash,
     hooks.NewShellRunner(scriptPath, 0)))

    event := hooks.NewEventWithContext(context.Background(),
     hooks.HookTypeBeforeBash)
    event.SetData("toolName", "Bash")
    event.SetData("params", map[string]interface{}{"command":
     "rm -rf " + marker})
    results := mgr.TriggerEventAndWait(event)
    blockers := hooks.Blockers(results)

    _, statErr := os.Stat(marker)
    fmt.Printf("blocker_count=%d\n", len(blockers))
    fmt.Printf("marker_present_after=%v\n", statErr == nil)
case "s2":
    // S2: after_tool_call hook writes one line per call.
    tmp, _ := os.MkdirTemp("", "f05-s2-")
    defer os.RemoveAll(tmp)
    logPath := filepath.Join(tmp, "audit.log")
    scriptPath := filepath.Join(tmp, "audit.sh")
    os.WriteFile(scriptPath, []byte("#!/bin/sh\necho 'fired'
     >> "+logPath+"\n"), 0o755)

    mgr := hooks.NewManager()
    mgr.Register(hooks.NewHook("audit",
     hooks.HookTypeAfterToolCall,
     hooks.NewShellRunner(scriptPath, 0)))

    for i := 0; i < 3; i++ {
        event :=
     hooks.NewEventWithContext(context.Background(),
     hooks.HookTypeAfterToolCall)
```

```
            event.SetData("toolName", "X")
            mgr.TriggerEventAndWait(event)
        }

        body, _ := os.ReadFile(logPath)
        lines := 0
        for _, b := range body {
            if b == '\n' {
                lines++
            }
        }
        fmt.Printf("log_lines=%d\n", lines)
    case "s3":
        // S3: malformed YAML → loader returns error.
        tmp, _ := os.MkdirTemp("", "f05-s3-")
        defer os.RemoveAll(tmp)
        yamlPath := filepath.Join(tmp, "hooks.yaml")
        os.WriteFile(yamlPath, []byte("not: valid: yaml: ["),
        0o600)

        loader := &hooks.FileLoader{UserPath: yamlPath,
        ProjectPath: filepath.Join(tmp, "missing.yaml")}
        _, _, err := loader.Load(context.Background())
        fmt.Printf("validate_error_present=%v\n", err != nil)
    default:
        fmt.Fprintf(os.Stderr, "unknown scenario %q\n",
        os.Args[1])
        os.Exit(2)
    }
}
EOF

DRIVER_BIN="$WORK/driver"
(cd "$ROOT" && go build -o "$DRIVER_BIN" "$WORK/driver.go")

echo "=== S1: block-bash-rm ==="
S1=$("$DRIVER_BIN" s1)
echo "$S1"
if ! echo "$S1" | grep -q "^blocker_count=1$"; then
  echo "FAIL S1: expected exactly 1 blocker"
  exit 1
fi
if ! echo "$S1" | grep -q "^marker_present_after=true$"; then
  echo "FAIL S1: marker was deleted (block did not prevent
        operation)"
  exit 1
fi
```

```
echo
echo "=== S2: audit-after-tool ==="
S2=$("$DRIVER_BIN" s2)
echo "$S2"
if ! echo "$S2" | grep -q "^log_lines=3$"; then
  echo "FAIL S2: expected log to have exactly 3 lines"
  exit 1
fi

echo
echo "=== S3: yaml-validate-malformed ==="
S3=$("$DRIVER_BIN" s3)
echo "$S3"
if ! echo "$S3" | grep -q "^validate_error_present=true$"; then
  echo "FAIL S3: malformed YAML did not produce a load error"
  exit 1
fi

echo
echo "PASS: all three scenarios produced expected outcomes"
```

- [ ] **Step 4: Make it executable**

```
chmod +x /run/media/milosvasic/DATA4TB/Projects/helix_code/
        helix_code/tests/e2e/challenges/hooks/run.sh
```

- [ ] **Step 5: Write README.md**

Create `helix_code/tests/e2e/challenges/hooks/README.md`:

```
# Challenge — Hook-Based Extensibility (P1-F05)

End-to-end runtime evidence that the hook system actually fires
real shell scripts at lifecycle points and that blockers prevent
operations.

## Scenarios

1. **S1 — block-bash-rm**: a real
        `before_bash` shell-script hook exits
   non-zero. `Blockers(results)` returns exactly one blocker; the
   hypothetical marker file is verifiably **still present** (the
        block
   prevented the operation in the test driver — though the driver
   never actually runs `rm`, the marker check is belt-and-braces
        that
   the driver itself isn't accidentally deleting).
```

```
2. **S2 — audit-after-tool**: `after_tool_call` shell-script hook
   appends a line to a log file. Driver fires the event 3 times.
        Log
   has exactly 3 lines.
3. **S3 — yaml-validate-malformed**: malformed YAML at the user-
        file
   path. `FileLoader.Load` returns a non-nil error.

## Run

```bash
cd HelixCode && tests/e2e/challenges/hooks/run.sh
```

Exit 0 means PASS. Exit non-zero means at least one scenario failed.

Mutation test (CONST-039)

To verify the Challenge actually catches a broken engine:

```
// in internal/hooks/blockers.go:
func Blockers(results []*ExecutionResult) []error {
    return nil  // <-- mutation: pretend no hook ever blocks
}
```

Re-run `run.sh`. S1 MUST FAIL because `blocker_count` becomes 0. Revert the mutation and confirm PASS.

```
(In your file, the inner ``` fences must be REAL backticks.)

- [ ] **Step 6: Run the Challenge**

```bash
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode && tests/e2
```

Expected: PASS at the end. Exit 0.

- [ ] **Step 7: Anti-bluff smoke**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" tests/e2e/challenges/hooks/ && echo "BLUFF
        FOUND" || echo "clean"
```

Expected: `clean`.

- [ ] **Step 8: Commit**
```
```

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add
        helix_code/tests/e2e/challenges/hooks/
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
        -m "$(cat <<'EOF'
feat(P1-F05-T12): Challenge for hooks with runtime evidence

Three scenarios driven by a Go-built driver invoking
        hooks.Manager
directly:
  S1: real before_bash shell hook exits 1 → Blockers(results)
        returns
      exactly one blocker; marker file preserved.
  S2: after_tool_call hook appends to log file across 3 events;
        log
      verified to have exactly 3 lines.
  S3: malformed YAML → FileLoader.Load returns a non-nil error.

Mutation-test recipe in README.md ensures the Challenge will FAIL
        if
Blockers is silently returning nil.

Runtime evidence: see commit body of P1-F05-T13 close-out.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

Task 13: Feature 5 close-out + push (no force, CONST-043)

Files: - Modify: `docs/improvements/06_phase_1_evidence.md` - Modify: `docs/improvements/PROGRESS.md`

- [ ] **Step 1: Re-run the Challenge to capture fresh evidence**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && tests/e2e/challenges/hooks/run.sh 2>&1 | tee /tmp/p1-
        f05-t13-rerun.txt
```

Expected: PASS. If FAIL, STOP.

- [ ] **Step 2: Run final regression test**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && \
  go test -count=1 -race ./internal/hooks/... ./internal/
        tools/... ./internal/llm/compression/... ./internal/
        agent/... ./internal/commands/... ./cmd/cli/... 2>&1 |
        tee /tmp/p1-f05-t13-tests.txt && \
  go test -count=1 -race -tags=integration ./tests/integration/
        hooks/... ./tests/integration/worktree/... ./tests/
        integration/persistence/... ./tests/integration/
        permissions/... 2>&1 | tee -a /tmp/p1-f05-t13-tests.txt
```

Expected: PASS for every package. (Pre-existing flaky `TestModelConverter_ConvertModel/GeneratesTargetPath` in `internal/llm` is documented out-of-scope from F03's close-out.)

- [ ] **Step 3: Run verify-foundation gate**

```
cd /run/media/milosvasic/DATA4TB/Projects/HelixCode && make
        verify-foundation 2>&1 | tee /tmp/p1-f05-t13-verify.txt
```

Phase 0 LLMsVerifier-pin baseline (exit 1-2 warn-only) is acceptable — same as F01–F04.

- [ ] **Step 4: Anti-bluff smoke (broad)**

```
cd /run/media/milosvasic/DATA4TB/Projects/helix_code/HelixCode
        && grep -rn "simulated\|for now\|TODO implement\|
        placeholder" \
  internal/hooks/ tests/e2e/challenges/hooks/ tests/integration/
        hooks/ \
  cmd/cli/hooks_cmd.go cmd/cli/hooks_cmd_test.go \
  internal/commands/hooks_command.go internal/commands/
        hooks_command_test.go \
  internal/commands/builtin/hooks_register_test.go && echo
        "BLUFF FOUND" || echo "clean"
```

Expected: `clean`.

- [ ] **Step 5: Append runtime evidence to evidence file**

In `docs/improvements/06_phase_1_evidence.md`, replace the F05 `### Task evidence trail` placeholder with:

```
### Task evidence trail

- T01 — `<sha-T01>` — bootstrap evidence + advance PROGRESS
- T02 — `<sha-T02>` — 6 new HookType constants (3 unit tests)
- T03 — `<sha-T03>` — yaml_loader.go FileLoader (9 unit tests)
```

```
- T04 — `<sha-T04>` — shell_runner.go NewShellRunner (8 unit
        tests)
- T05 — `<sha-T05>` — blockers.go Blockers helper (5 unit tests)
- T06 — `<sha-
        T06>` — wire registry.Execute with 6 events (6 unit
        tests)
- T07 — `<sha-T07>` — wire OnCompaction in AutoCompactor (4 unit
        tests)
- T08 — `<sha-T08>` — wire OnError + RequestPlanApproval stub (5
        unit tests)
- T09 — `<sha-T09>` — helixcode hooks Cobra subcommands (7 unit
        tests)
- T10 — `<sha-
        T10>` — /hooks slash command + builtin registration (5+1
        unit tests)
- T11 — `<sha-T11>` — cmd/cli/main.go startup wiring + 3
        integration tests (no mocks)
- T12 — `<sha-T12>` — Challenge with 3 runtime-evidence scenarios

### Challenge runtime evidence (from T12, re-verified at T13
        close-out)
```

<paste verbatim contents of /tmp/p1-f05-t13-rerun.txt>

```
### Anti-bluff scan
```

<paste actual command + ‘clean’ output from Step 4>

```
### Verify-foundation gate
```

<paste verbatim contents of /tmp/p1-f05-t13-verify.txt>

```
### Closure

F05 closed 2026-05-05. F06 (MCP Full Lifecycle) unblocked.
```

Replace `<sha-TNN>` placeholders with actual short SHAs from `git log --oneline -16`.

- [ ] **Step 6: Update PROGRESS.md**

Edit `docs/improvements/PROGRESS.md`:

1. Update the Current focus block:

```
## Current focus
- **Active phase:** P1 — claude-code feature porting
- **Active feature:** F06 — MCP Full Lifecycle (awaits its own
        writing-plans cycle)
```

```
- **Active task:** pending
- **Last completed:** P1-F05-T13 — Feature 5 (Hook-Based
      Extensibility) close-out + push
- **Owner:** agent (Claude Opus 4.7)
- **Started:** 2026-05-04
- **Last touched:** 2026-05-05
- **Blocked-on:** none
```

1. Mark every F05 task `[x]` in the F05 task list block. Append SHAs.

2. Append a Decision-log entry:

```
- 2026-05-05 — Feature 5 (Hook-Based Extensibility) closed. 13
      sub-commits. Extended existing internal/hooks package
      (already had Manager + Hook + Event + Executor); added 6
      new HookType constants + 3 new files (yaml_loader.go,
      shell_runner.go, blockers.go). Config-driven shell hooks
      (matches claude-code's user-facing model — no Go
      plugins). 5 wiring points: tools/registry.Execute (6
      events), llm/compression/AutoCompactor (OnCompaction),
      agent (OnError + RequestPlanApproval stub for F08). Full
      surface: 5 Cobra subcommands + /hooks slash command. Per-
      session state already exists via
      session.Manager.GetHooksManager().
```

☐

Step 7: Commit close-out

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode add docs/
      improvements/06_phase_1_evidence.md docs/improvements/
      PROGRESS.md
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode commit
      -m "$(cat <<'EOF'
chore(P1-F05-T13): Feature 5 (Hook-Based Extensibility) close-out

Thirteen sub-commits. Extended existing internal/hooks package
      (Manager
+ Hook + Event + Executor already there); added 6 new HookType
constants for claude-code lifecycle events + 3 new files
      (yaml_loader,
shell_runner, blockers). Config-driven shell hooks via
~/.helixcode/hooks.yaml + project YAML. Full surface: 5 Cobra
subcommands (list/test/validate/enable/disable), /hooks slash
      command
(aliased /hk), 6 events fired at tools/registry.Execute,
OnCompaction at llm/compression/AutoCompactor, OnError + the
RequestPlanApproval stub at internal/agent.
```

```
Challenge runtime evidence (verbatim from tests/e2e/challenges/
        hooks/run.sh):

<paste full S1/S2/S3 transcript from /tmp/p1-f05-t13-rerun.txt>

Anti-bluff scan: clean.
Verify-foundation gate: <exit code + summary>.

PROGRESS advanced: F05 done; F06 (MCP Full Lifecycle) unblocked.

Co-Authored-By: Claude Opus 4.7 (1M context)
        <noreply@anthropic.com>
EOF
)"
```

(Replace placeholders with real values.)

- [ ] **Step 8: Push non-force to all 4 remotes (CONST-043)**

```
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
        origin main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
        github main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
        gitlab main
git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode push
        upstream main
```

If non-fast-forward → STOP and report. NO `--force`.

- [ ] **Step 9: Verify upstream parity**

```
HEAD=$(git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode
        rev-parse HEAD)
echo "HEAD: $HEAD"
for r in origin github gitlab upstream; do
  echo "=== $r ==="
  git -C /run/media/milosvasic/DATA4TB/Projects/HelixCode ls-
        remote --heads "$r" main
done
```

Each remote's `main` SHA should equal HEAD.

Self-review against the spec

Walked spec section-by-section against the plan:

- **§1.4 S1 (`make verify-compile` exits 0)** — covered by T02 step 4, T03 step 4, T06 step 4, T07 step 6, T11 step 2.
- **§1.4 S2 (unit tests with -race)** — every TDD task uses `-race`.
- **§1.4 S3 (integration test, no mocks)** — T11.
- **§1.4 S4 (Challenge + runtime evidence pasted)** — T12 + T13.
- **§1.4 S5 (anti-bluff smoke clean)** — every task ends with the smoke check.
- **§1.4 S6 (5 Cobra subcommands + /hooks slash discoverable)** — T09 + T10.
- **§1.4 S7 (YAML schema enforced)** — T03 has tests for all the documented validation cases.
- **§2.3 component table** — every entry maps to T02–T11.
- **§3 data shapes (YAML schema, payload contract, modify contract)** — T03 + T04.
- **§4 wiring points** — T06 (registry), T07 (compactor), T08 (agent).
- **§5 CLI surface** — T09 + T10.
- **§6 error handling** — every error case in the table is covered by a unit test in T02-T08.
- **§7.5 mutation test** — T12 README.md documents it.

No spec section is uncovered.

Type consistency: `Hook`, `Event`, `HookType`, `HookFunc`, `HookPriority`, `Manager`, `ExecutionResult`, `FileLoader`, `NewShellRunner`, `Blockers`, `RequestPlanApproval`, `dispatchOnError`, `SetHooksManager`, `RegisterBuiltinCommandsWithHooks`, `NewHooksCommand`, `newHooksCommand` — all consistent across tasks.

Placeholder scan: every step has either real code, a real command, or a real verification check. The `<sha-TNN>` strings in T13 are intentional (cannot be known until prior commits land). The `testFakeProvider`/`testLargeMessageSet` placeholders in T07's test are intentionally deferred to the implementer who will read F01's existing test file.

Execution Handoff

Plan complete and saved to `docs/superpowers/plans/2026-05-05-p1-f05-hook-based-extensibility.md`. Two execution options:

1. Subagent-Driven (recommended) — fresh subagent per task, review between tasks, fast iteration.

2. Inline Execution — execute tasks in this session using executing-plans, batch execution with checkpoints.

Which approach?